

pytop Kullanım Kılavuzu

Nokta-Küme Topolojisi ve Metrik Uzaylar

Kadirhan Polat

2026

İçindekiler

Ön Söz	vi
I Ön Koşullar	1
1 pytop'a Hızlı Giriş	2
1.1 Kurulum ve Import	2
1.2 Uzay Nesneleri	2
1.3 Result Tipi	2
1.4 Temel Uzaylar Turu	3
1.5 Özel Topoloji: make_topology	3
1.6 Tag Sistemi	4
1.7 n Nokta Üzerindeki Topoloji Sayısı	4
2 Önerme Mantığı	5
2.1 Önerme ve Doğruluk Değeri	5
2.2 Mantıksal Bağlaçlar	5
2.3 Algoritmalar	6
2.3.1 Doğruluk Tablosu Üretimi	6
2.3.2 Tautoloji Denetimi	6
2.4 pytop API	6
2.5 Örnekler	7
2.5.1 Doğruluk Tablosu	7
2.5.2 Tautolojiler — De Morgan ve Karşıt	7
2.5.3 Niceleyiciler	8
2.5.4 Topoloji Uygulaması — T0 ve T1	8
2.6 Alıştırmalar	9
3 Küme Teorisi ve Fonksiyon Temelleri	10
3.1 Konu	10
3.2 Teoremler	10
3.3 Algoritmalar	11
3.3.1 Güç Kümesi — $O(2^{ A })$	11
3.3.2 Denklik Bağıntısı Doğrulama — $O(R \cdot A)$	11
3.4 pytop API	11
3.5 Örnekler	12
3.6 Alıştırmalar	13

II	Nokta-Küme Topolojisi	14
4	Topolojik Uzaylar	15
4.1	Konu	15
4.1.1	Sezgisel Giriş	15
4.1.2	Formal Tanım	15
4.1.3	Temel Örnekler	16
4.1.4	Baz ve Alt-Baz	16
4.1.5	Sonlu ve Sonsuz Uzaylar	17
4.2	Teoremler	17
4.3	Algoritmalar	18
4.3.1	Bazdan Topoloji Üretimi	18
4.3.2	İz Sürme: Küçük Bir Girdiyle Adım Adım	19
4.3.3	Alt-Bazdan Topoloji Üretimi	19
4.4	pytop API	19
4.4.1	Sınıflar	19
4.4.2	Oluşturucular	19
4.4.3	Tag Sistemi	20
4.4.4	Tag Gerekçeleri	20
4.5	Örnekler	20
4.5.1	Örnek 1: Sierpiński Uzayı	20
4.5.2	Örnek 2: Ayrık Topoloji	21
4.5.3	Örnek 3: <code>make_topology</code> ile Manuel İnşa	22
4.5.4	Örnek 4: Alexandrov Zincir Uzayı	23
4.5.5	Örnek 5: Bazdan Topoloji Üretimi	23
4.5.6	Örnek 6: Gerçek Doğru	24
4.6	Alıştırmalar	24
5	Yüklemler ve Altküme Operatörleri	26
5.1	Konu	26
5.1.1	Altküme Yüklemeleri	26
5.1.2	Altküme Operatörleri	26
5.1.3	Komşuluk Sistemi	26
5.2	Teoremler	27
5.3	Algoritmalar	27
5.4	pytop API	28
5.5	Örnekler	28
5.5.1	Örnek 1: Kapanış ve İç — Sierpiński Uzayı	28
5.5.2	Örnek 2: Türev Kümesi	29
5.5.3	Örnek 3: Komşuluk Sistemi	29
5.6	Alıştırmalar	30
6	Ayrılma Aksiyomları	31
6.1	Konu	31
6.2	Teoremler	31
6.3	Algoritmalar	32
6.4	pytop API	32
6.5	Örnekler	32
6.5.1	Örnek 1: Sierpiński — T_0 ✓, T_1 ×	32

6.5.2	Örnek 2: Kosonlu $\mathbb{N}$ — $T_1 \checkmark$, $T_2 \times$	33
6.5.3	Örnek 3: Ayrılma Zinciri — Ayrık	33
6.6	Alıştırmalar	33
7	Kompaktlık	34
7.1	Konu	34
7.1.1	Kompaktlık Tanımı	34
7.1.2	Kompaktlık Varyantları	34
7.2	Teoremler	34
7.3	Algoritmalar	35
7.3.1	Sonlu Uzayda Kompaktlık	35
7.3.2	analyze_compactness_variants — Tag Tabanlı Çıkarım	35
7.4	pytop API	35
7.5	Örnekler	35
7.6	Alıştırmalar	36
8	Bağlantılılık	38
8.1	Konu	38
8.1.1	Bağlantılılık Kavramları	38
8.1.2	Clopen Ayrılma	38
8.2	Teoremler	38
8.3	Algoritmalar	39
8.3.1	Sonlu Bağlantılılık — $O(\tau ^2)$	39
8.4	pytop API	39
8.5	Örnekler	39
8.6	Alıştırmalar	40
9	Sayılabilirlik Aksiyomları	41
9.1	Konu	41
9.1.1	Sayılabilirlik Aksiyomları	41
9.2	Teoremler	41
9.3	Algoritmalar	42
9.3.1	Sonlu Uzayda Sayılabilirlik	42
9.3.2	Sonsuz Uzayda: Tag-Tabanlı Çıkarım	42
9.4	pytop API	42
9.5	Örnekler	42
9.6	Alıştırmalar	43
10	Süreklili Fonksiyonlar ve Homeomorfizmalar	44
10.1	Konu	44
10.2	Teoremler	44
10.3	Algoritmalar	45
10.3.1	is_continuous_finite_map — $O(\tau_Y \cdot X)$	45
10.4	pytop API	45
10.5	Örnekler	45
10.6	Alıştırmalar	46

11 Alt Uzay ve Çarpım Topolojisi	47
11.1 Konu	47
11.2 Teoremler	47
11.3 Algoritmalar	48
11.3.1 finite_subspace — $O(\tau \cdot A)$	48
11.3.2 binary_product — $O(\tau_X \cdot \tau_Y)$	48
11.4 pytop API	48
11.5 Örnekler	48
11.6 Alıştırmalar	49
12 Bölüm Topolojisi	50
12.1 Konu	50
12.2 Teoremler	50
12.3 Algoritmalar	51
12.3.1 quotient_set — $O(X ^2 \cdot \alpha(X))$	51
12.3.2 finite_quotient_contract — $O(X + P)$	51
12.4 pytop API	51
12.5 Örnekler	51
12.6 Alıştırmalar	53
13 Başlangıç ve Son Topoloji	54
13.1 Başlangıç Topolojisi	54
13.2 Algoritmalar	54
13.2.1 Başlangıç Topolojisi İnşası	54
13.2.2 Son Topoloji İnşası	55
13.3 pytop API	55
13.4 Örnekler	56
13.4.1 Tek Harita ile Başlangıç Topolojisi	56
13.4.2 İki Harita — Topoloji İncelir	56
13.4.3 Altuzay = Başlangıç	56
13.4.4 Çarpım = Başlangıç	57
13.4.5 Son Topoloji — Manuel Hesap	57
13.4.6 Bölüm = Son Topoloji	58
13.5 Alıştırmalar	58
III Metrik Uzaylar	59
14 Metrik Uzaylar	60
14.1 Konu	60
14.1.1 Metrik Aksiyomları	60
14.1.2 Temel Kavramlar	60
14.1.3 Standart Metrikler	60
14.2 Teoremler	61
14.3 Algoritmalar	61
14.3.1 validate_metric — $O(X ^3)$	61
14.3.2 open_ball — $O(X)$	61
14.4 pytop API	62
14.5 Örnekler	62

14.6	Alıřtırmalar	63
15	Metrik Tamlık ve Kompaktlık	64
15.1	Konu	64
15.1.1	Cauchy Dizisi ve Tamlık	64
15.1.2	Tamamen Sınırlılık	64
15.1.3	Metrik Kompaktlık Karakterizasyonu	64
15.2	Teoremler	65
15.3	Algoritmalar	65
15.3.1	Sonlu Metrikte <code>is_complete</code> ve <code>is_totally_bounded</code>	65
15.3.2	<code>metric_compactness_check</code>	65
15.4	pytop API	65
15.5	Örnekler	65
15.6	Alıřtırmalar	66
16	Metrik Fonksiyonlar ve Sözleşmeler	67
16.1	Konu	67
16.1.1	Metrik Fonksiyon Sınıfları	67
16.1.2	Lipschitz Sabiti	67
16.2	Teoremler	67
16.3	Algoritmalar	68
16.3.1	<code>classify_finite_metric_map</code> — $O(X ^2)$	68
16.4	pytop API	68
16.5	Örnekler	68
16.6	Alıřtırmalar	69
A	Alıřtırma Çözümleri	70
	Dizin	71

Ön Söz

Bu kılavuz `pytop` paketinin nokta-küme topolojisi ve metrik uzaylar modüllerini kapsamlı biçimde tanıtmaktadır. Her bölüm aynı yapıyı izler:

1. Teorik giriş ve formal tanımlar
2. Temel teoremler (kanıt iskeletleriyle)
3. Algoritmalar (sözde kod ve karmaşıklık)
4. `pytop` API
5. Çalışan örnekler
6. Alıştırmalar

Örnekler aynı zamanda `docs/user_guide/python/` altındaki Python script'lerinde ve `docs/user_guide/notebook/` altındaki Jupyter defterlerinde çalıştırılabilir biçimde mevcuttur.

Kısım I

Ön Koşullar

Bölüm 1

pytop'a Hızlı Giriş

pytop kurulumu, temel uzay nesneleri, `Result` tipi ve tag sistemi — kılavuzun geri kalanını okumadan önce bu bölümü çalıştırın.

1.1 Kurulum ve Import

```
1 pip install -e . # git kokundan
```

Her bölümde ihtiyac duyulan semboller doğrudan `pytop`'tan import edilir.

1.2 Uzay Nesneleri

Kurucu	Topoloji	Sonuç
<code>discrete_topology(*pts)</code>	Her altküme açık	Ayrık
<code>indiscrete_topology(*pts)</code>	Yalnız $\emptyset$ ve X açık	İndiscrete
<code>sierpinski_space()</code>	$\{\emptyset, \{1\}, \{0, 1\}\}$	Sierpiński
<code>make_topology(carrier, *open_sets)</code>	Kullanıcı tanımlı	Özel

Tablo 1.1: Temel uzay kurucuları

```
1 d = discrete_topology(0, 1, 2)
2 print("carrier:", sorted(d.carrier))
3 print("topoloji boyutu:", len(d.topology))
4 print("etiketler:", sorted(d.tags))
```

```
carrier: [0, 1, 2]
topoloji boyutu: 8
etiketler: ['discrete', 'finite', 'hausdorff', 'metrizable', 'normal', 'regular']
```

1.3 Result Tipi

`pytop`'un çoğu yüklemi bir `Result` nesnesi döner.

```

1 s = sierpinski_space()
2 r = is_t2(s)
3 print(".status:", r.status)           # 'true' | 'false' | 'unknown'
4 print(".value:", r.value)
5 print(".mode:", r.mode)
6 print(".justification:", r.justification)

```

```

.status: false
.value: hausdorff
.mode: exact
.justification: ['The explicit finite topology fails hausdorff.']

```

.status her zaman 'true', 'false' veya 'unknown' dizesidir — Python bool'u değil.

1.4 Temel Uzaylar Turu

```

1 spaces = {
2     "Ayrik D(0,1,2)": discrete_topology(0, 1, 2),
3     "Indiscrete I(0,1,2)": indiscrete_topology(0, 1, 2),
4     "Sierpinski S": sierpinski_space(),
5 }
6 for name, sp in spaces.items():
7     print(name,
8           is_compact(sp).status,
9           is_connected(sp).status,
10          is_t2(sp).status)

```

```

Ayrik D(0,1,2)      true  false  true
Indiscrete I(0,1,2) true  true   false
Sierpinski S       true  true   false

```

1.5 Özel Topoloji: make_topology

```

1 X = [1, 2, 3, 4]
2 tau = [set(), {1,2}, {3,4}, {1,2,3,4}]
3 fts = make_topology(X, *tau)
4 print("topoloji boyutu:", len(fts.topology))
5 print("kompakt:", is_compact(fts).status)
6 print("T2:", is_t2(fts).status)

```

```

topoloji boyutu: 4
kompakt: true
T2: false

```

1.6 Tag Sistemi

```
1 print("Ayrik:      ", sorted(discrete_topology(0,1,2).tags))
2 print("Indiscrete:", sorted(indiscrete_topology(0,1,2).tags))
3 print("Sierpinski:", sorted(sierpinski_space().tags))
```

```
Ayrik:      ['discrete', 'finite', 'hausdorff', 'metrizable', 'normal', 'regular']
Indiscrete: ['finite', 'indiscrete']
Sierpinski: ['finite', 'sierpinski']
```

Etiketler yüklem fonksiyonlarının karar mekanizmasını hızlandırır: `is_t2(d)` açık küme taraması yerine `'hausdorff'` in `d.tags` kontrolüyle sonuç verir.

1.7 n Nokta Üzerindeki Topoloji Sayısı

```
1 for n in range(1, 6):
2     print(f"n={n}: {count_topologies_on_n_points(n)}")
```

```
n =1: 1
n =2: 4
n =3: 29
n =4: 355
n =5: 6942
```

Topoloji sayısı hızla büyür; sonlu uzayların tüm topolojileri üzerinde kaba kuvvet taraması yalnızca küçük n için pratiktir.

Bölüm 2

Önerme Mantığı

Matematiksel önerme (proposition), doğru ya da yanlış olabilen bir ifadedir. Bu bölüm `pytop.logic` modülünün sağladığı `Proposition`, bağlaçlar ve sonlu niceleyicileri tanıtır; ardından ayrılma aksiyomlarını bu araçlarla nasıl ifade edebileceğimizi gösterir.

2.1 Önerme ve Doğruluk Değeri

Tanım 2.1 (Önerme). Bir **önerme** (proposition), kesin bir doğruluk değerine sahip (doğru veya yanlış) matematiksel bir ifadedir.

Listing 2.1: Temel önermeler

```
1 from pytop import (  
2     Proposition,  
3     negate, conjunction, disjunction, implies, iff,  
4     for_all, there_exists, unique_exists,  
5 )  
6  
7 p = Proposition("p", True)  
8 q = Proposition("q", False)  
9  
10 print(f"p : {p.truth_value}")  
11 print(f"q : {q.truth_value}")
```

```
p : True  
q : False
```

2.2 Mantıksal Bağlaçlar

```
1 print("neg(p)  :", negate(p).truth_value)  
2 print("p and q :", conjunction(p, q).truth_value)  
3 print("p or q  :", disjunction(p, q).truth_value)  
4 print("p -> q  :", implies(p, q).truth_value)  
5 print("p <-> q :", iff(p, q).truth_value)  
6 print("p <-> p :", iff(p, p).truth_value)
```

Fonksiyon	Sembol	Anlam
negate(p)	$\neg p$	p değil
conjunction(p,q)	$p \wedge q$	p ve q
disjunction(p,q)	$p \vee q$	p veya q
implies(p,q)	$p \rightarrow q$	p ise q
iff(p,q)	$p \leftrightarrow q$	p ancak ve ancak q

Tablo 2.1: pytop.logic bağlaç fonksiyonları

```

neg(p) : False
p and q : False
p or q : True
p -> q : False
p <-> q : False
p <-> p : True

```

`implies(p, q)` yalnızca $p = \text{True}$, $q = \text{False}$ durumunda yanlışır; diğer üç kombinasyonda boş doğruluk (vacuous truth) nedeniyle doğrudur.

2.3 Algoritmalar

2.3.1 Doğruluk Tablosu Üretimi

Algorithm 1 Doğruluk Tablosu (İki Değişkenli)

Require: Bağlaç fonksiyonu f

Ensure: Dört satırlı doğruluk tablosu

```

1: for  $(t_p, t_q) \in \{(T, T), (T, F), (F, T), (F, F)\}$  do
2:    $p \leftarrow \text{Proposition}("p", t_p)$ 
3:    $q \leftarrow \text{Proposition}("q", t_q)$ 
4:   Yazdır  $(t_p, t_q, f(p, q).truth\_value)$ 
5: end for

```

2.3.2 Tautoloji Denetimi

Bir $\phi(p_1, \dots, p_n)$ formülü tautoloji iff tüm 2^n atama için doğrudur — $O(2^n \cdot |\phi|)$.

2.4 pytop API

Listing 2.2: logic modülü importu

```

1 from pytop import (
2     Proposition,
3     negate, conjunction, disjunction, implies, iff,
4     for_all, there_exists, unique_exists,
5 )

```

Niceleyiciler:

```

1 for_all(carrier, predicate)      # tüm x: P(x)
2 there_exists(carrier, predicate) # bazı x: P(x)
3 unique_exists(carrier, predicate) # tam bir x: P(x)

```

2.5 Örnekler

2.5.1 Doğruluk Tablosu

```

1 pairs = [(True, True), (True, False), (False, True), (False, False)]
2 print(f"{'p':>5} {'q':>5} {'neg_p':>5} {'p&q':>5} {'p|q':>5} {'p
   ->q':>6} {'p<->q':>6}")
3 for tv_p, tv_q in pairs:
4     pp = Proposition("p", tv_p)
5     qq = Proposition("q", tv_q)
6     row = (pp.truth_value, qq.truth_value,
7            negate(pp).truth_value, conjunction(pp, qq).truth_value,
8            disjunction(pp, qq).truth_value,
9            implies(pp, qq).truth_value, iff(pp, qq).truth_value)
10    print(" ".join(f"{str(v):>5}" for v in row))

```

p	q	neg_p	p&q	p q	p->q	p<->q
True	True	False	True	True	True	True
True	False	False	False	True	False	False
False	True	True	False	True	True	False
False	False	True	False	False	True	True

2.5.2 Tautolojiler — De Morgan ve Karşıt

Teorem 2.2 (De Morgan Yasaları).

$$\neg(p \wedge q) \leftrightarrow \neg p \vee \neg q$$

$$\neg(p \vee q) \leftrightarrow \neg p \wedge \neg q$$

Teorem 2.3 (Karşıt (Contrapositive)). $(p \rightarrow q) \leftrightarrow (\neg q \rightarrow \neg p)$

```

1 def check_tautology(name, func):
2     ok = all(func(Proposition("p", tp), Proposition("q", tq)).
3               truth_value
4               for tp, tq in [(True, True), (True, False), (False, True), (
5                               False, False)])
6     print(f"{name}: {'tautoloji' if ok else 'DEGIL'}")
7
8 check_tautology("neg(p&q) <-> neg_p|neg_q",
9                 lambda p, q: iff(negate(conjunction(p, q)), disjunction(negate(p),
10                                negate(q))))
11
12 check_tautology("(p->q) <-> (neg_q->neg_p)",
13                 lambda p, q: iff(implies(p, q), implies(negate(q), negate(p))))

```

```
neg(p&q) <-> neg_p|neg_q: tautoloji
(p->q) <-> (neg_q->neg_p): tautoloji
```

2.5.3 Niceleyiciler

```
1 X = [1, 2, 3, 4, 5]
2 print("for_all(X, x>0)      :", for_all(X, lambda x: x > 0))
3 print("for_all(X, x>2)      :", for_all(X, lambda x: x > 2))
4 print("there_exists(X, x>4)  :", there_exists(X, lambda x: x > 4))
5 print("unique_exists(X, x==3):", unique_exists(X, lambda x: x == 3))
```

```
for_all(X, x>0)      : True
for_all(X, x>2)      : False
there_exists(X, x>4)  : True
unique_exists(X, x==3): True
```

2.5.4 Topoloji Uygulaması — T0 ve T1

Tanım 2.4 (T0 — Kolmogorov). $\forall x \neq y \in X, \exists$ açık $U : (x \in U, y \notin U)$ veya $(y \in U, x \notin U)$.

Tanım 2.5 (T1 — Fréchet). $\forall x \neq y \in X, \exists$ açık $U : x \in U, y \notin U$.

```
1 from pytop import sierpinski_space, discrete_topology,
   indiscrete_topology
2
3 def is_t0_logic(space):
4     pts = list(space.carrier)
5     opens = list(space.topology)
6     return for_all(
7         [(x, y) for x in pts for y in pts if x != y],
8         lambda pair: there_exists(
9             opens, lambda U: (pair[0] in U) != (pair[1] in U)
10        )
11    )
12
13 def is_t1_logic(space):
14     pts = list(space.carrier)
15     opens = list(space.topology)
16     return for_all(
17         [(x, y) for x in pts for y in pts if x != y],
18         lambda pair: there_exists(
19             opens, lambda U: pair[0] in U and pair[1] not in U
20        )
21    )
22
23 for name, sp in [("Sierpinski", sierpinski_space()),
24                 ("Discrete", discrete_topology(0, 1)),
25                 ("Indiscrete", indiscrete_topology(0, 1))]:
```

26

```
print(f"{name}    T0={is_t0_logic(sp)}    T1={is_t1_logic(sp)}")
```

Sierpinski	T0=True	T1=False
Discrete	T0=True	T1=True
Indiscrete	T0=False	T1=False

2.6 Alıştırmalar

1. Beş önerme değeriyle tam bir doğruluk tablosu oluşturun; `implies` kaç (p, q) çiftinde `False` döner?
2. `unique_exists([0,1,2,3,4], predicate)` değerini `True` yapan üç farklı koşul yazın.
3. T2 (Hausdorff) aksiyomunu `for_all` ve `there_exists` kullanarak kodlayın: $\forall x \neq y, \exists$ açık $U, V : x \in U, y \in V, U \cap V = \emptyset$. Sierpiński, discrete ve indiscrete için test edin.
4. (Teori) `implies(p, q)` neden yalnızca $p = T, q = F$ durumunda yanlıştır? Boş doğruluk (vacuous truth) kavramını açıklayın.
5. (Teori) De Morgan yasalarını önerme mantığı için ispatlayın: $\neg(p \wedge q) \leftrightarrow \neg p \vee \neg q$.

Bölüm 3

Küme Teorisi ve Fonksiyon Temelleri

pytop'un tüm modülleri küme teorisi ve fonksiyon kavramları üzerine kuruludur. Bu bölüm kılavuzun geri kalanında varsayılan ön koşulları pytop API'siyle pekiştirir.

3.1 Konu

Tanım 3.1 (Altküme ve Güç Kümesi). $A \subseteq B \iff \forall a \in A, a \in B$. **Güç kümesi:** $\mathcal{P}(A) = \{S : S \subseteq A\}$, $|\mathcal{P}(A)| = 2^{|A|}$.

Tanım 3.2 (Denklik Bağıntısı). A üzerinde $R \subseteq A \times A$ bağıntısı **denklik bağıntısıdır** iff: yansımali ($\forall a, (a, a) \in R$), simetrik ($(a, b) \in R \Rightarrow (b, a) \in R$), geçişli ($(a, b), (b, c) \in R \Rightarrow (a, c) \in R$). Denklik sınıfı $[a] = \{b \in A : (a, b) \in R\}$.

Tanım 3.3 (Fonksiyon Türleri). $f : A \rightarrow B$ için:

- **Enjeksiyon:** $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$
- **Sürjeksiyon:** $\forall b \in B, \exists a \in A : f(a) = b$
- **Bijeksiyon:** Enjeksiyon + Sürjeksiyon

3.2 Teoremler

Teorem 3.4 (De Morgan). $(A \cup B)^c = A^c \cap B^c$ ve $(A \cap B)^c = A^c \cup B^c$.

Teorem 3.5 (Güç Kümesi Boyutu). $|A| = n \Rightarrow |\mathcal{P}(A)| = 2^n$.

Teorem 3.6 (Denklik Sınıfları Bölüntü Oluşturur). $\sim$ denklik bağıntısı ise $\{[a] : a \in A\}$ A 'nın bir bölüntüsüdür: sınıflar örtüşmez ve birleşimleri A 'dır.

Teorem 3.7 (Ön-Görüntü Özellikleri). $f : A \rightarrow B$ ve $S_1, S_2 \subseteq B$ için: $f^{-1}(S_1 \cup S_2) = f^{-1}(S_1) \cup f^{-1}(S_2)$ ve $f^{-1}(S_1 \cap S_2) = f^{-1}(S_1) \cap f^{-1}(S_2)$.

Algorithm 2 PowerSet(A)

```

1: result  $\leftarrow \{\emptyset\}$ 
2: for each  $a \in A$  do
3:   result  $\leftarrow$  result  $\cup \{S \cup \{a\} : S \in \text{result}\}$ 
4: end for
5: return result

```

Algorithm 3 IsEquivalence(A, R)

```

1: for each  $a \in A$  do
2:   if  $(a, a) \notin R$  then return False
3:   end if
4: end for
5: for each  $(a, b) \in R$  do
6:   if  $(b, a) \notin R$  then return False
7:   end if
8: end for
9: for each  $(a, b) \in R, (b, c) \in R$  do
10:  if  $(a, c) \notin R$  then return False
11:  end if
12: end for
13: return True

```

3.3 Algoritmalar

3.3.1 Güç Kümesi — $O(2^{|A|})$

3.3.2 Denklik Bağıntısı Doğrulama — $O(|R| \cdot |A|)$

3.4 pytop API

```

1 from pytop import (
2     make_set, power_set,
3     set_union, set_intersection, set_difference,
4     is_subset, is_proper_subset, equal_sets,
5     make_relation, is_equivalence_relation,
6     identity_relation, inverse_relation, relation_profile,
7     normalize_finite_map_data,
8     is_injective_finite_map, is_surjective_finite_map,
9     is_bijective_finite_map,
10    image_of_subset_finite, preimage_of_subset_finite,
11 )

```

```

make_set(*elements)  $\rightarrow$  frozenset
power_set(values)  $\rightarrow$  set[frozenset]
make_relation(carrier, *pairs)  $\rightarrow$  set[tuple]
relation_profile(carrier, relation)  $\rightarrow$  dict
normalize_finite_map_data(domain, codomain, mapping)  $\rightarrow$  FiniteMapData

```

3.5 Örnekler

Örnek 5.1 — Küme İşlemleri

```

1 A = make_set(1, 2, 3)
2 B = make_set(2, 3, 4)
3 print("A union B:", sorted(set_union(A, B)))
4 print("A inter B:", sorted(set_intersection(A, B)))
5 print("A diff B:", sorted(set_difference(A, B)))
6 print("{2,3} subset A:", is_subset({2, 3}, A))

```

```

A union B: [1, 2, 3, 4]
A inter B: [2, 3]
A diff B: [1]
{2,3} subset A: True

```

Örnek 5.2 — Güç Kümesi

```

1 P = power_set([1, 2, 3])
2 print("boyut:", len(P)) # 8 = 2^3

```

```
boyut: 8
```

Örnek 5.3 — Denklik Bağıntısı

```

1 carrier = [1, 2, 3, 4]
2 rel_eq = make_relation(carrier,
3     (1,1),(2,2),(3,3),(4,4),(1,3),(3,1),(2,4),(4,2))
4 print("denklik:", is_equivalence_relation(carrier, rel_eq))
5 prof = relation_profile(carrier, rel_eq)
6 print("ıyansimalı:", prof['is_reflexive'])
7 print("simetrik:", prof['is_symmetric'])
8 print("gecisli:", prof['is_transitive'])
9
10 # Gecissiz (1,2),(2,3) var ama (1,3) yok
11 rel_bad = make_relation(carrier,
12     (1,1),(2,2),(3,3),(4,4),(1,2),(2,1),(2,3),(3,2))
13 print("gecissiz denklik mi:", is_equivalence_relation(carrier,
14     rel_bad))

```

```

denklik: True
ıyansimalı: True
simetrik: True
gecisli: True
gecissiz denklik mi: False

```

Örnek 5.4 — Fonksiyon Türleri

```

1 f = normalize_finite_map_data(
2   [1,2,3], ['a','b','c'], {1:'a', 2:'b', 3:'c'})
3 print("bijeksiyon:", is_bijective_finite_map(f))
4
5 g = normalize_finite_map_data(
6   [1,2,3], ['x','y'], {1:'x', 2:'x', 3:'y'})
7 print("g enjeksiyon:", is_injective_finite_map(g))
8 print("g surjeksiyon:", is_surjective_finite_map(g))

```

```

bijeksiyon: True
g enjeksiyon: False
g surjeksiyon: True

```

Örnek 5.5 — Görüntü ve Ön-Görüntü

```

1 f = normalize_finite_map_data(
2   [1,2,3,4], ['a','b','c','d'], {1:'a',2:'b',3:'c',4:'d'})
3 print("f({1,2}):", sorted(image_of_subset_finite(f, [1, 2])))
4 print("f^-1({b,c}):", sorted(preimage_of_subset_finite(f, ['b','c'])))

```

```

f({1,2}): ['a', 'b']
f^-1({b,c}): [2, 3]

```

3.6 Alıştırmalar

Kodlama

- K1. $A = \{1, 2, 3, 4, 5\}$, $B = \{3, 4, 5, 6, 7\}$ için $A \cup B$, $A \cap B$, $A \setminus B$ 'yi hesaplayın.
- K2. $\{1, 2, 3, 4\}$ için güç kümesini üretin; $|\mathcal{P}| = 16$ olduğunu doğrulayın.
- K3. $R = \{(i, j) : 0 \leq i, j \leq 2\}$ (tam çarpım) üzerinde `relation_profile` çalıştırın ve denklik olduğunu gösterin.

Teori

- T1. $A \subseteq B \Leftrightarrow A \cap B = A$ olduğunu ispatlayın.
- T2. $f : A \rightarrow B$ ve $g : B \rightarrow C$ enjeksiyon ise $g \circ f$ de enjeksiyondur.

Kısım II

Nokta-Küme Topolojisi

Bölüm 4

Topolojik Uzaylar

4.1 Konu

4.1.1 Sezgisel Giriş

Topoloji, bir kümedeki noktaların birbirine “yakın” olup olmadığını, aralarındaki sürekliliği ve şekilsel özellikleri, mesafe kavramına gerek duymadan inceleyen matematiksel bir disiplindir. Bunu mümkün kılan temel araç *açık küme* kavramıdır.

Düşünelim: gerçek doğrudaki (a, b) aralığı “açık”tır; her noktasının çevresinde küçük bir açık aralık daha bulunur. Bu sezgiyi soyutlayarak *herhangi* bir küme üzerinde topoloji tanımlamak mümkündür.

Sezgi

Bir şehir haritasında “yakınlık”ı sokak mesafesiyle ölçebilirsiniz; ama “hangi mahalleler birbirine komşu?” sorusu mesafe olmadan da anlamlıdır. Topoloji tam bunu yapar: mesafeyi unuttur, yalnızca “hangi kümeler bir noktanın çevresini oluşturur?” bilgisini tutar. Matematiksel karşılığı: τ ailesi her noktanın “çevre” kavramını açık kümeler aracılığıyla kodlar; süreklilik ve yakınsama gibi kavramlar yalnız τ ’ya başvurularak tanımlanır.

4.1.2 Formal Tanım

Tanım 4.1 (Topolojik Uzay). Bir X kümesi ve $\tau \subseteq \mathcal{P}(X)$ ailesi verilsin. (X, τ) bir **topolojik uzay**, τ ise X üzerinde bir **topoloji** olarak adlandırılır, eğer aşağıdaki üç aksiyom sağlanıyorsa:

$$(T1) \quad \emptyset \in \tau \text{ ve } X \in \tau$$

$$(T2) \quad U, V \in \tau \Rightarrow U \cap V \in \tau$$

$$(T3) \quad \{U_\alpha\}_{\alpha \in I} \subseteq \tau \Rightarrow \bigcup_{\alpha \in I} U_\alpha \in \tau$$

τ ’nın elemanları **açık küme** olarak adlandırılır.

Aksiyomların her biri ayrı bir iş görür: (T1) uzayın tamamının ve boş kümenin her zaman “gözlemlenebilir” olmasını garanti eder; (T2) iki gözlemin kesişiminin de gözlem olmasını sağlar — sonlu kesişimle sınırlı kalması bilinçli bir tercihtir (bkz. Teorem 4.4);

(T3) keyfi birleşime izin vererek küçük çevrelerden büyük çevre kurma işlemini serbest bırakır.

Karşı-örnek

$X = \{1, 2, 3\}$ üzerinde $\sigma = \{\emptyset, \{1\}, \{2\}, X\}$ ailesi bir topoloji *değildir*: $\{1\} \cup \{2\} = \{1, 2\} \notin \sigma$ olduğundan (T3) ihlal edilir. (T1) ve (T2) sağlansa bile tek bir eksik birleşim aileyi topoloji olmaktan çıkarır. Bu ailenin `make_topology`'ye verilince ne olduğunu Örnekler bölümündeki “Dikkat” kutusunda göreceğiz.

4.1.3 Temel Örnekler

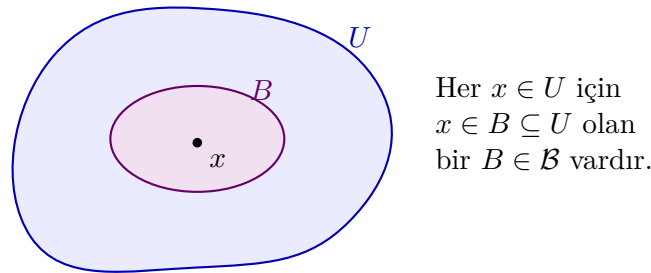
Topoloji	Tanım	Açık kümeler
Ayrık (discrete)	Her alt küme açık	$\tau = \mathcal{P}(X)$
İndirgenmiş (indiscrete)	Yalnızca $\emptyset$ ve X açık	$\tau = \{\emptyset, X\}$
Kosonlu (cofinite)	U açık $\iff U = \emptyset$ veya $X \setminus U$ sonlu	—
Sierpiński	$X = \{0, 1\}$, $\tau = \{\emptyset, \{1\}, X\}$	3 açık küme

Tablo 4.1: Standart topoloji örnekleri

4.1.4 Baz ve Alt-Baz

Tanım 4.2 (Baz). $\mathcal{B} \subseteq \tau$ ailesi, her $U \in \tau$ ve her $x \in U$ için $x \in B \subseteq U$ sağlayan bir $B \in \mathcal{B}$ varsa τ 'ya **baz** denir.

Şekil 4.1 koşulu resmeder: açık kümenin her noktası, tamamen o kümenin içinde kalan bir baz elemanı ile sarılır.



Şekil 4.1: Baz koşulunun geometrisi: U açık kümesinin her x noktası için $x \in B \subseteq U$ sağlayan bir $B \in \mathcal{B}$ vardır. Topolojinin tüm açıkları bu tür “yapı taşları”nın birleşimleridir.

Neden önemli?

Baz, büyük bir topolojiyi küçük bir çekirdekle temsil etme aracıdır. Gerçek doğrunun standart topolojisi sayılamaz çoklukta açık küme içerir; ama tümü, sayılabilir $\{(a, b) : a < b, a, b \in \mathbb{Q}\}$ bazından üretilir. `pytop` tarafında `topology_from_basis` tam bu ilkeyle çalışır: yalnız baz elemanlarını verirsiniz, kapanışı kütüphane hesaplar (Algoritma 4).

Baz Koşulları:

$$(B1) \bigcup \mathcal{B} = X$$

$$(B2) B_1, B_2 \in \mathcal{B} \text{ ve } x \in B_1 \cap B_2 \text{ ise bir } B_3 \in \mathcal{B} \text{ vardır: } x \in B_3 \subseteq B_1 \cap B_2$$

Tanım 4.3 (Alt-Baz). $\mathcal{S} \subseteq \mathcal{P}(X)$ ailesi; sonlu kesişimleri baz, onların birleşimleri ise τ 'yu oluşturur.

4.1.5 Sonlu ve Sonsuz Uzaylar

pytop'ta iki temel sınıf bulunur:

- **FiniteTopologicalSpace:** Taşıyıcı (X) sonlu; topoloji açıkça frozenset'ler koleksiyonu olarak saklanır. Tüm hesaplamalar tam ve kesindir.
- **Sonsuz türevler:** Taşıyıcı simgesel ('R', 'N'); topology =None; etiket tabanlı sembolik çıkarım yapılır.

4.2 Teoremler

Teorem 4.4 (Aksiyomların Yeterliliği). (X, τ) bir topolojik uzay olsun. $T1$ – $T3$ aksiyomları, sonlu herhangi bir açık küme ailesinin kesişiminin τ 'da yer aldığını garanti eder:

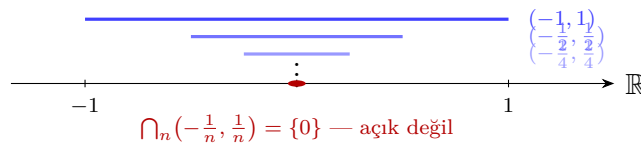
$$U_1, \dots, U_n \in \tau \Rightarrow \bigcap_{i=1}^n U_i \in \tau.$$

Not: Sonsuz kesişim gerekmez. Gerçek doğrudaki $\bigcap_{n=1}^{\infty} (-\frac{1}{n}, \frac{1}{n}) = \{0\}$ açık değildir.

Rehberli Kanıt. Strateji: n üzerinde tümevarım; tek araç (T2).

1. $n = 1$: $U_1 \in \tau$ zaten verili.
2. $n = k$ için doğru varsayalım: $V = \bigcap_{i=1}^k U_i \in \tau$.
3. $n = k + 1$: $\bigcap_{i=1}^{k+1} U_i = V \cap U_{k+1}$ olup (T2) gereği τ 'dadır.

Sonsuz kesişimde 3. adım çalışmaz: tümevarım yalnız sonlu n 'lere ulaşır. Şekil 4.2 bunun gerçek bir başarısızlık olduğunu gösterir: her sonlu kesişim açıkken limit $\{0\}$ açık değildir. $\square$



Şekil 4.2: (T2)'nin sonlu kesişimle sınırlı olmasının nedeni: iç içe $(-\frac{1}{n}, \frac{1}{n})$ açık aralıklarının sonsuz kesişimi tek noktaya çöker ve $\{0\}$ standart topolojide açık değildir.

Teorem 4.5 (Baz Teoremi). $\mathcal{B} \subseteq \mathcal{P}(X)$ ailesi (B1)–(B2) koşullarına sağlıyorsa,

$$\tau_{\mathcal{B}} = \{U \subseteq X : \forall x \in U, \exists B \in \mathcal{B}, x \in B \subseteq U\}$$

bir topoloji olur ve $\mathcal{B}$ bu topolojiye baz oluşturur.

Rehberli Kanıt. Strateji: $\tau_{\mathcal{B}}$ 'nin üç aksiyomu sağladığını, her adımda yalnız (B1)–(B2)'yi kullanarak doğrularız.

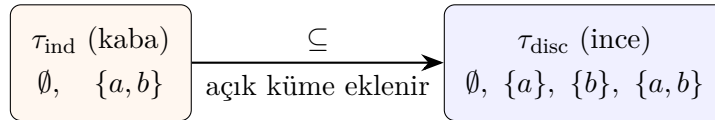
1. **(T1):** $\emptyset \in \tau_{\mathcal{B}}$, çünkü “her $x \in \emptyset$ için...” koşulu boş yere doğrudur. $X \in \tau_{\mathcal{B}}$: $x \in X$ verilsin; (B1) gereği $x \in \bigcup \mathcal{B}$, yani $x \in B \subseteq X$ olan bir $B \in \mathcal{B}$ vardır.
2. **(T2):** $U, V \in \tau_{\mathcal{B}}$ ve $x \in U \cap V$ olsun. Tanım gereği $x \in B_U \subseteq U$ ve $x \in B_V \subseteq V$ bulunur. (B2), $x \in B_3 \subseteq B_U \cap B_V \subseteq U \cap V$ veren bir B_3 sağlar; x keyfi olduğundan $U \cap V \in \tau_{\mathcal{B}}$.
3. **(T3):** $x \in \bigcup_{\alpha} U_{\alpha}$ ise $x \in U_{\alpha_0}$ olan bir indis vardır; U_{α_0} 'ın tanımından gelen B , $x \in B \subseteq U_{\alpha_0} \subseteq \bigcup_{\alpha} U_{\alpha}$ koşulunu da sağlar.
4. $\mathcal{B}$ 'nin $\tau_{\mathcal{B}}$ 'ye baz olması, $\tau_{\mathcal{B}}$ 'nin tanımının Tanım 4.2'teki koşulu birebir içermesindendir.

□

Bu teorem, `topology_from_basis`'in döndürdüğü ailenin gerçekten topoloji olduğunun matematiksel güvencesidir; fonksiyon ayrıca (B1)–(B2)'yi sağlamayan girdiyi `BasisConstructionError` ile reddeder.

Teorem 4.6 (Karşılaştırma). X üzerinde iki topoloji $\tau_1 \subseteq \tau_2$ ise τ_1 **kaba (coarser)**, τ_2 **ince (finer)** topoloji olarak adlandırılır. Ayırık topoloji en ince, indirgenmiş topoloji en kaba topolojidir.

Rehberli Kanıt. Herhangi bir τ için $\tau \subseteq \mathcal{P}(X) = \tau_{\text{disc}}$ topoloji tanımının kendisinden gelir (aksiyom gerekmez); (T1) aksiyomu da $\{\emptyset, X\} = \tau_{\text{ind}} \subseteq \tau$ verir. □



Şekil 4.3: Aynı $X = \{a, b\}$ üzerinde iki uç: indirgenmiş topoloji (kaba) ile ayırık topoloji (ince). Açık küme eklemek topolojiyi inceltir; tüm topolojiler bu iki uç arasında yaşar.

`pytop` bu uçları `tags` ile işaretler: `indiscrete_topology` nesnesi `indiscrete`, `discrete_topology` nesnesi `discrete` etiketi taşır.

4.3 Algoritmalar

4.3.1 Bazdan Topoloji Üretimi

Problem: X ve $\mathcal{B} \subseteq \mathcal{P}(X)$ verilsin; $\tau_{\mathcal{B}}$ 'yi hesapla.

Karmaşıklık:

- En kötü durum: $O(|\mathcal{B}| \cdot 2^{|\mathcal{B}|})$ (tüm alt-ailelerin birleşimi)
- Pratik (çakışmayan baz elemanları): $O(|\tau| \cdot |\mathcal{B}|)$
- Uzay: $O(|\tau|)$ açık küme saklar

Algorithm 4 Bazdan Topoloji Üretimi**Require:** X (taşıyıcı), $\mathcal{B}$ (baz ailesi)**Ensure:** τ (kapalı topoloji)

```

1:  $\tau \leftarrow \{\emptyset, X\}$ 
2: for each non-empty  $\mathcal{S} \subseteq \mathcal{B}$  do
3:    $\tau.add(\bigcup_{B \in \mathcal{S}} B)$ 
4: end for
5: CloseUnderIntersection( $\tau$ ) return  $\tau$ 

```

Algorithm 5 Kesişim Altında Kapatma

```

1:  $changed \leftarrow \mathbf{true}$ 
2: while  $changed$  do
3:    $changed \leftarrow \mathbf{false}$ 
4:   for each  $U, V \in \tau$  do
5:     if  $U \cap V \notin \tau$  then
6:        $\tau.add(U \cap V)$ ;  $changed \leftarrow \mathbf{true}$ 
7:     end if
8:   end for
9: end while

```

4.3.2 İz Sürme: Küçük Bir Girdiyle Adım Adım

$X = \{1, 2, 3\}$, $\mathcal{B} = \{\{1\}, \{2, 3\}\}$ girdisiyle Algoritma 4:

4.3.3 Alt-Bazdan Topoloji Üretimi

$\mathcal{S}$ verilsin; önce $\mathcal{S}$ 'nin sonlu kesişimlerinden baz oluştur, ardından Algoritma 4'yi uygula.

Karmaşıklık: $O(|\mathcal{S}|^k \cdot 2^{|\mathcal{S}|^k})$ — k : kesişim derinliği.

4.4 pytop API**4.4.1 Sınıflar**

Listing 4.1: Temel sınıf importları

```
1 from pytop import TopologicalSpace, FiniteTopologicalSpace
```

Alanlar: `carrier` (altta yatan küme), `topology` (açık kümeler), `tags` (özellik etiketleri), `metadata` (ek bilgiler).

4.4.2 Oluşturucular

Listing 4.2: Topoloji oluşturucu importları

```
1 from pytop import (
2     make_topology, discrete_topology, indiscrete_topology,
3     cofinite_topology, sierpinski_space,
4     topology_from_basis, topology_from_subbasis,
5 )
```

Adım	$\mathcal{S}$ (alt-aile)	Eklenen birleşim	τ (o ana dek)
0	—	—	$\{\emptyset, X\}$
1	$\{\{1\}\}$	$\{1\}$	$\{\emptyset, X, \{1\}\}$
2	$\{\{2, 3\}\}$	$\{2, 3\}$	$\{\emptyset, X, \{1\}, \{2, 3\}\}$
3	$\{\{1\}, \{2, 3\}\}$	$\{1\} \cup \{2, 3\} = X$	değişmez
4	kesişim kapanışı	$\{1\} \cap \{2, 3\} = \emptyset$	değişmez

Tablo 4.2: İz sürme: $|\tau| = 4$; döngü yeni küme üretmeyince durur.

Sınıf	Taşıyıcı	Topoloji
TopologicalSpace	Any	Any (None olabilir)
FiniteTopologicalSpace	sonlu frozenset	frozenset[frozenset]

Tablo 4.3: Temel uzay sınıfları

4.4.3 Tag Sistemi

pytop nesneleri tags kümesi aracılığıyla topolojik özellikler taşır. Örnek etiketler: "finite", "discrete", "t0", "t1", "hausdorff", "compact", "connected", "metric".

4.4.4 Tag Gerekçeleri

Etiketler, kurucunun inşa anında *garanti ettiği* gerçeklerdir:

Etiketler *eksiksiz değildir*: `cofinite_topology('a','b','c')` aslında ayrıktır ve Hausdorff'tur ($2^3 = 8$ açık), ama `discrete/hausdorff` etiketi taşımaz — `is_t2` yüklemi yine `true` döner. Kesin sorgu için Bölüm 6'daki `is_*` yüklemelerini kullanın; etiket, yüklemine yerine geçmez.

4.5 Örnekler

4.5.1 Örnek 1: Sierpiński Uzayı

Listing 4.3: Sierpiński uzayı

```

1 from pytop import sierpinski_space
2
3 s = sierpinski_space()
4 print("Taşıyıcı:", s.carrier)
5 print("Topoloji:", sorted(str(t) for t in s.topology))
6 print("Etiketler:", sorted(s.tags))

```

Listing 4.4: Çıktı

```

1 Taşıyıcı: frozenset({0, 1})
2 Topoloji: ['frozenset()', 'frozenset({0, 1})', 'frozenset({1})']
3 Etiketler: ['compact', 'connected', 'finite', 't0']

```

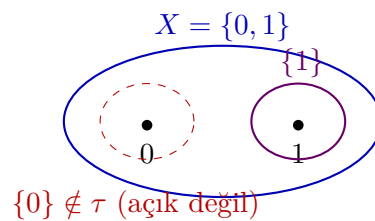
Fonksiyon	İmza	Açıklama
make_topology	(carrier, *open_sets)	Açık küme listesiyle inşa
discrete_topology	(*elements)	Ayrık topoloji
indiscrete_topology	(*elements)	İndirgenmiş topoloji
cofinite_topology	(*elements)	Kosonlu topoloji
sierpinski_space	()	Sierpiński uzayı
topology_from_basis	(carrier, basis)	Bazdan topoloji üret
topology_from_subbasis	(carrier, subbasis)	Alt-bazdan topoloji üret

Tablo 4.4: Topoloji oluşturu fonksiyonlar

Kurucu	Etiketler	Gerekçe
sierpinski_space()	compact, connected, finite, t0	Sonlu $\Rightarrow$ kompakt; $\{1\}$ tek y lü ayırır $\Rightarrow$ T0 (T1 değil); ay parçalanış yok $\Rightarrow$ bağlantılı
discrete_topology(1,2,3)	discrete, finite, hausdorff, metrizable, normal, regular	Her tekil açık $\Rightarrow$ tüm ayrı aksiyomları; 0–1 metriği ay topolojiyi üretir
indiscrete_topology(1,2,3)	compact, connected, finite, indiscrete	Açık yalnız $\emptyset$ ve X : parçala maz
cofinite_topology('a','b','c')	cofinite, compact, finite, t1	Tekiller kapalı $\Rightarrow$ T1
make_topology(...), finite_chain_space(n)	finite	Özellik çıkarımı yapılmaz; nız sonluluk işaretlenir

Tablo 4.5: Pilot bölümdeki kurucuların etiket gerekçeleri

Ne oldu? Çıktıdaki üç satırı tek tek okuyalım. Tasiyici satırı $X = \{0,1\}$ 'i verir. Topoloji satırındaki üç küme tam olarak Tanım 4.1'in istediği yapıdır: $\emptyset$ ve X (T1), tek ek açık küme $\{1\}$; $\{1\} \cap X = \{1\}$ ve $\{1\} \cup X = X$ zaten listede olduğundan (T2)–(T3) de sağlanır. Etiketler satırında t0 var ama t1 yok: 1'i içerip 0'ı dışlayan açık küme vardır ($\{1\}$), fakat 0'ı içerip 1'i dışlayan açık küme yoktur — T0 sağlanıp T1'in sağlanmamasının kaynağı budur (Bölüm 6, Örnek 1'de yüklemelerle test edilir). Şekil 4.4 açık küme yapısını gösterir.



Şekil 4.4: Sierpiński uzayının açıkları: $\emptyset$, $\{1\}$ ve X . Kesikli bölge $\{0\}$ 'ın açık *olmadığını* vurgular; bu asimetri uzayı T0 yapar ama T1 yapmaz.

4.5.2 Örnek 2: Ayrık Topoloji

Listing 4.5: Ayrık topoloji

```

1 from pytop import discrete_topology
2
3 d = discrete_topology(1, 2, 3)
4 print("|tau| =", len(d.topology))
5 print("Etiketler:", sorted(d.tags))

```

Listing 4.6: Çıktı

```

1 |tau| = 8
2 Etiketler: ['discrete', 'finite', 'hausdorff', 'metrizable', 'normal',
3            , 'regular']

```

Ne oldu? $n = 3$ eleman için $|\tau| = 2^3 = 8$: her alt küme açıktır. Her tekil küme açık olduğundan herhangi iki nokta kendi tekil komşuluklarıyla ayrılır — **hausdorff**, **normal**, **regular** etiketlerinin kaynağı budur. **metrizable**, 0–1 metriğinden gelir: $d(x, y) = 1$ ($x \neq y$) metriği tam olarak ayrık topolojiyi üretir. Listede **compact** yok; oysa uzay sonlu olduğu için kompakttır ve **is_compact** yüklemi **true** döner — fark için Tag Gereçekleri tablosuna bakın.

4.5.3 Örnek 3: make_topology ile Manuel İnşa

Listing 4.7: Manuel topoloji inşası

```

1 from pytop import make_topology
2
3 sp = make_topology({1, 2, 3}, {1}, {2, 3})
4 print("|tau| =", len(sp.topology))
5 print("Acik kumeler:", sorted(str(t) for t in sp.topology))

```

Listing 4.8: Çıktı

```

1 |tau| = 4
2 Acik kumeler: ['frozenset()', 'frozenset({1, 2, 3})',
3              'frozenset({1})', 'frozenset({2, 3})']

```

Ne oldu? **make_topology** verilen $\{1\}$ ve $\{2, 3\}$ açıklarına yalnızca $\emptyset$ ve X 'i ekledi; $|\tau| = 4$. Bu örnekte sonuç geçerli bir topolojidir, çünkü verilen iki küme ayrıktır: birleşimleri X , kesişimleri $\emptyset$ zaten ailededir. **make_topology** bu kapanışları *hesaplamaz* — aşağıdaki kutu, kapalı olmayan bir aile verildiğinde ne olduğunu gösterir.

Dikkat – sık hata

make_topology verdiğiniz aileye yalnızca $\emptyset$ ve X 'i ekler; birleşim/kesişim kapanışını **hesaplamaz** ve aksiyomları **denetlemez**. Aşağıdaki çağrıda $\{1\} \cup \{2\} = \{1, 2\}$ ailede yoktur; dönen nesne (T3)'ü ihlal eden, geçersiz bir “topoloji”dir. Kapanışın hesaplanmasını ve ailenin denetlenmesini istiyorsanız **topology_from_basis** kullanın — o, baz koşullarını sağlamayan aileyi **BasisConstructionError** ile reddeder.

Listing 4.9: Kapanış hesaplanmaz — sorumluluk kullanıcıda

```

1 from pytop import make_topology, topology_from_basis
2
3 sessiz = make_topology({1, 2, 3}, {1}, {2})    # denetlemez!
4 print("make_topology:", sorted(sorted(t) for t in sessiz.topology))
5
6 try:
7     topology_from_basis({1, 2, 3}, [{1}, {2}]) # B1 ihlali
8 except Exception as e:
9     print("topology_from_basis:", type(e).__name__)

```

Listing 4.10: Çıktı

```

make_topology: [[], [1], [1, 2, 3], [2]]
topology_from_topology: BasisConstructionError

```

4.5.4 Örnek 4: Alexandrov Zincir Uzayı

Listing 4.11: Zincir uzayı

```

1 from pytop import finite_chain_space
2
3 c = finite_chain_space(3)
4 print("Tasiyici:", c.carrier)
5 print("Topoloji:", sorted(str(t) for t in c.topology))

```

Listing 4.12: Çıktı

```

1 Tasiyici: (1, 2, 3)
2 Topoloji: ['set()', '{1, 2, 3}', '{1, 2}', '{1}']

```

Ne oldu? Çıktıdaki açıklar tam bir “önek merdiveni”dir: $\emptyset \subset \{1\} \subset \{1, 2\} \subset \{1, 2, 3\}$. Alexandrov zincir uzayında 1 her boş olmayan açıktaki yer alır (“en açık” nokta), 3 yalnız X ’te görünür. Öneklerin kesişimi ve birleşimi yine önek olduğundan (T2)–(T3) otomatik sağlanır.

4.5.5 Örnek 5: Bazdan Topoloji Üretimi

Listing 4.13: Bazdan topoloji

```

1 from pytop import topology_from_basis
2
3 b = [{1}, {2, 3}, {4}]
4 ts = topology_from_basis({1, 2, 3, 4}, b)
5 print("Baz:", [set(x) for x in b])
6 print("|tau| =", len(ts.topology))
7 print("Topoloji:", sorted(str(t) for t in ts.topology))

```

Listing 4.14: Çıktı

```

1 Baz: [{1}, {2, 3}, {4}]
2 |tau| = 8
3 Topoloji: ['set()', '{1, 2, 3, 4}', '{1, 2, 3}', '{1, 4}',
4           '{1}', '{2, 3, 4}', '{2, 3}', '{4}']

```

Ne oldu? Baz $\{\{1\}, \{2, 3\}, \{4\}\}$ bir bölüntüdür: elemanları ikiye ikiye ayrılır. (B2) koşulu boş yere sağlanır (kesişimler boş olduğundan kontrol edilecek nokta yoktur) ve topoloji tüm alt-aile birleşimlerinden oluşur: $2^3 = 8$ açık küme. Çıktıdaki 8 küme, üç baz elemanının 2^3 alt kümesinin birleşimleriyle birebir eşleşir.

4.5.6 Örnek 6: Gerçek Doğru

Listing 4.15: Gerçek doğru

```

1 from pytop import real_line_metric
2
3 rl = real_line_metric()
4 print("Tasiyici:", rl.carrier)
5 print("Etiketler:", sorted(rl.tags))

```

Listing 4.16: Çıktı

```

1 Tasiyici: R
2 Etiketler: ['complete', 'connected', 'first_countable', 'hausdorff',
3           'infinite', 'lindelof', 'metric', 'not_compact',
4           'path_connected', 'second_countable', 'separable', 't0',
5           't1', 'uncountable']

```

Ne oldu? Tasiyici: R satırı taşıyıcının *simgesel* olduğunu söyler: gerçek doğrunun noktaları bellekte tutulmaz, `topology = None`’dır. Tüm topolojik bilgi etiketlerde kodlanmıştır: `connected`, `hausdorff`, `second_countable` gibi olumlu özellikler yanında `not_compact` gibi olumsuz bilgi de taşınır (Heine–Borel: $\mathbb{R}$ sınırlı değildir). Sonlu uzaylardaki “hesapla ve doğrula” yaklaşımının yerini burada “bilinen teoremleri etiketle” yaklaşımı alır; metrik tarafı Bölüm 14’te derinleşir.

4.6 Alıştırmalar

Kodlama Alıştırmaları

K1. `cofinite_topology('a', 'b', 'c')` ile üç noktalı kosonlu uzayı kurun; topolojisini ve etiketlerini yazdırın, `is_t1` ve `is_t2` ile test edin. Gözlem: sonlu bir kümede kosonlu topoloji ayrık topolojiyle çıkarışır. T1 olup Hausdorff olmayan bir örnek için taşıyıcının neden sonsuz olması gerektiğini bir cümleyle açıklayın (krş. Bölüm 6, Örnek 2).

İpucu: $|\tau| = 2^3$ çıkacak; anahtar, Bölüm 6’daki “Sonlu T1 $\iff$ Ayrık” teoremidir. (çözüm)

K2. `topology_from_subbasis({1,2,3,4}, [{1,2},{3,4},{2,3}])` çağrısının ürettiği topolojinin kaç açık küme içerdiğini bulun ve açık kümeleri listeleyin.

İpucu: Önce alt-baz çiftlerinin kesişimlerini elle listeleyin ($\{2\}$, $\{3\}$, $\emptyset$); sonra birleşimleri sayın. (çözüm)

K3. `finite_chain_space(5)` ile bir Alexandrov-5 zinciri oluşturun; topolojinin kaç açık küme içerdiğini ve hangi elemanın “en açık” nokta olduğunu bulun.

İpucu: Açıklar örnek yapısındadır; “en açık” nokta her boş olmayan açıkta bulunandır. (çözüm)

Teori Alıştırmaları

T1. $X = \{1, 2, 3\}$ üzerinde kaç farklı topoloji tanımlanabilir? Bu sayıyı T1–T3 aksiyomlarını doğrudan kontrol ederek hesaplamak yerine neden baz teoremini kullanmak daha pratiktir?

İpucu: Aday aile sayısı $2^{2^3} = 256$ ’dır; cevap 29. Baz teoremi adayları üretken küçük ailelere indirger. (çözüm)

T2. Ayrık topolojinin her zaman diğer topolojilerden daha ince, indirgenmiş topolojinin ise daha kaba olduğunu kanıtlayın. Kanıt için T1–T3 aksiyomlarından hangisi gereklidir?

İpucu: Ayrıklık için aksiyom gerekmez ($\tau \subseteq \mathcal{P}(X)$ tanım gereğidir); indirgenmişlik için yalnız (T1) yeter. (çözüm)

Bölüm 5

Yüklemler ve Altküme Operatörleri

5.1 Konu

5.1.1 Altküme Yüklemeleri

(X, τ) bir topolojik uzay ve $A \subseteq X$ olsun.

Yüklem	Tanım
Açık (open)	$A \in \tau$
Kapalı (closed)	$X \setminus A \in \tau$
Clopen	$A \in \tau$ ve $X \setminus A \in \tau$
Yoğun (dense)	Her boş olmayan $U \in \tau$ için $U \cap A \neq \emptyset$
Hiçbir yerde-yoğun	$\text{int}(\overline{A}) = \emptyset$

Tablo 5.1: Altküme yüklemeleri

5.1.2 Altküme Operatörleri

Operatör	Gösterim	Tanım
Kapanış	$\overline{A} = \text{cl}(A)$	A 'yı içeren en küçük kapalı küme
İç	$A^\circ = \text{int}(A)$	A 'ya dahil en büyük açık küme
Sınır	$\partial A = \text{bd}(A)$	$\overline{A} \setminus A^\circ$
Türev kümesi	$A' = d(A)$	A 'nın birikme noktaları kümesi

Tablo 5.2: Altküme operatörleri

Birikme noktası: $x \in A' \iff$ her $U \ni x$ açık için $U \cap (A \setminus \{x\}) \neq \emptyset$.

5.1.3 Komşuluk Sistemi

Tanım 5.1 (Komşuluk Sistemi). $x \in X$ için:

$$N(x) = \{N \subseteq X : \exists U \in \tau, x \in U \subseteq N\}$$

Komşuluk Aksiyomları (N1–N4):

- (N1) $x \in N$, her $N \in N(x)$ için
 (N2) $X \in N(x)$
 (N3) $N_1, N_2 \in N(x) \Rightarrow N_1 \cap N_2 \in N(x)$
 (N4) $N \in N(x), N \subseteq M \Rightarrow M \in N(x)$

5.2 Teoremler

Teorem 5.2 (Kuratowski Kapanış Aksiyomları). $\text{cl} : \mathcal{P}(X) \rightarrow \mathcal{P}(X)$ operatörü şu dört özelliği sağlar:

- (K1) $\text{cl}(\emptyset) = \emptyset$
 (K2) $A \subseteq \text{cl}(A)$
 (K3) $\text{cl}(\text{cl}(A)) = \text{cl}(A)$
 (K4) $\text{cl}(A \cup B) = \text{cl}(A) \cup \text{cl}(B)$

Bu dört özelliği sağlayan her operatör bir topoloji tanımlar.

Teorem 5.3 (İç-Kapanış Dualitesi). Her $A \subseteq X$ için:

$$A^\circ = X \setminus \text{cl}(X \setminus A), \quad \text{cl}(A) = X \setminus \text{int}(X \setminus A).$$

Teorem 5.4 (Komşuluk Sistemi Round-Trip). $N1-N_4$ 'ü sağlayan her $\{N(x)\}_{x \in X}$ ailesi, $\tau = \{U : \forall x \in U, U \in N(x)\}$ şeklinde tek bir topolojiyi geri üretir.

5.3 Algoritmalar

Algorithm 6 Sonlu Kapanış Hesabı

Require: $A \subseteq X, \tau$

Ensure: $\text{cl}(A)$

- 1: $\text{closed} \leftarrow \{X \setminus U : U \in \tau\}$
 - 2: $\text{result} \leftarrow X$
 - 3: **for** each $C \in \text{closed}$ **do**
 - 4: **if** $A \subseteq C$ **and** $|C| < |\text{result}|$ **then**
 - 5: $\text{result} \leftarrow C$
 - 6: **end if**
 - 7: **end for** **return** result
-

Karmaşıklık: $O(|\tau| \cdot |X|)$.

Karmaşıklık: $O(|X| \cdot |\tau|)$.

Algorithm 7 Türev Kümesi Hesabı**Require:** $A \subseteq X, \tau$ **Ensure:** $d(A)$

```

1:  $acc \leftarrow \emptyset$ 
2: for each  $x \in X$  do
3:   if her açık  $U \ni x$  için  $U \cap (A \setminus \{x\}) \neq \emptyset$  then
4:      $acc.add(x)$ 
5:   end if
6: end for return  $acc$ 

```

5.4 pytop API

Listing 5.1: Subset operatörleri importları

```

1 from pytop import (
2     analyze_predicate,
3     closure_of_subset, interior_of_subset, boundary_of_subset,
4     derived_set_of_subset,
5     is_open_subset, is_closed_subset, is_dense_subset,
6     is_nowhere_dense_subset,
7     neighborhood_system, character_at_point,
8     analyze_neighborhood_system,
9 )

```

Fonksiyon	İmza	Açıklama
analyze_predicate	(space, predicate, subset)	Yüklem sorgula
closure_of_subset	(space, subset)	Kapanış
interior_of_subset	(space, subset)	İç
boundary_of_subset	(space, subset)	Sınır
derived_set_of_subset	(space, subset)	Türev kümesi
neighborhood_system	(carrier, topology, point)	$N(x)$
character_at_point	(carrier, topology, point)	$\chi(x)$
analyze_neighborhood_system	(carrier, topology, point =None)	Tam analiz

Tablo 5.3: Altküme operatör fonksiyonları

Not: neighborhood_system ve ilgili fonksiyonlar carrier (liste) ve topology (liste) alır. Space nesnesi için: list(space.carrier), list(space.topology).

5.5 Örnekler

5.5.1 Örnek 1: Kapanış ve İç — Sierpiński Uzayı

```

1 from pytop import sierpinski_space, closure_of_subset,
2     interior_of_subset, boundary_of_subset
3 s = sierpinski_space()

```

```

4 print("cl({0}) =", closure_of_subset(s, {0}).value)
5 print("cl({1}) =", closure_of_subset(s, {1}).value)
6 print("int({0}) =", interior_of_subset(s, {0}).value)
7 print("int({1}) =", interior_of_subset(s, {1}).value)
8 print("bd({1}) =", boundary_of_subset(s, {1}).value)

```

Listing 5.2: Çıktı

```

1 cl({0}) = frozenset({0})
2 cl({1}) = frozenset({0, 1})
3 int({0}) = frozenset()
4 int({1}) = frozenset({1})
5 bd({1}) = frozenset({0})

```

$\{0\}$ 'ın kapanışı kendisidir: $\{0\} = X \setminus \{1\}$ zaten kapalıdır. $\{1\}$ 'in kapanışı X 'tir: $\{1\}$ kapalı değildir.

5.5.2 Örnek 2: Türev Kümesi

```

1 from pytop import sierpinski_space, derived_set_of_subset,
  is_nowhere_dense_subset
2
3 s = sierpinski_space()
4 print("d({0}) =", derived_set_of_subset(s, {0}).value)
5 print("d({1}) =", derived_set_of_subset(s, {1}).value)
6 print("{0} nowhere dense? =", is_nowhere_dense_subset(s, {0}).status)

```

Listing 5.3: Çıktı

```

1 d({0}) = frozenset()
2 d({1}) = frozenset({0})
3 {0} nowhere dense? = true

```

$d(\{1\}) = \{0\}$: 0'ı içeren her $\{0, 1\}$, $\{1\} \setminus \{0\} = \{1\}$ ile kesişir.

5.5.3 Örnek 3: Komşuluk Sistemi

```

1 from pytop import sierpinski_space, neighborhood_system,
  character_at_point
2
3 s = sierpinski_space()
4 carrier = list(s.carrier)
5 topology = list(s.topology)
6 print("N(0) =", neighborhood_system(carrier, topology, 0).value)
7 print("N(1) =", neighborhood_system(carrier, topology, 1).value)
8 print("chi(0) =", character_at_point(carrier, topology, 0).value)
9 print("chi(1) =", character_at_point(carrier, topology, 1).value)

```

Listing 5.4: Çıktı

```

1 N(0) = [['0', '1']]
2 N(1) = [['1'], ['0', '1']]

```

```

3 chi(0) = 1
4 chi(1) = 2

```

$N(0) = \{X\}$: 0 yalnızca X 'te açık. $N(1) = \{\{1\}, X\}$: yerel baz büyüklükleri $\chi(0) = 1$, $\chi(1) = 2$.

5.6 Alıştırmalar

Kodlama Alıştırmaları

- K1.** İndirgenmiş iki-noktalı topolojide her noktanın komşuluk sistemini hesaplayın.
- K2.** $X = \{1, 2, 3, 4\}$, $\tau = \{\emptyset, \{1, 2\}, \{3, 4\}, X\}$ topolojisinde $\text{bd}(\{1, 2, 3\})$ ve $\text{cl}(\{1, 2, 3\})$ hesaplayın.
- K3.** `finite_chain_space(4)` zincirinde $\{1, 2\}$ 'nin iç, kapanış ve sınır kümelerini bulun.

Teori Alıştırmaları

- T1.** $A^\circ = X \setminus \text{cl}(X \setminus A)$ eşitliğini Kuratowski aksiyomlarını kullanarak kanıtlayın.
- T2.** A 'nın yoğun olması için gerek ve yeter koşulun $\text{cl}(A) = X$ olduğunu gösterin.

Bölüm 6

Ayrılma Aksiyomları

6.1 Konu

Ayrılma aksiyomları, bir topolojik uzaydaki noktaların ve kapalı kümelerin birbirinden açık kümeler aracılığıyla ne ölçüde “ayrılabilirliğini” ölçer.

Aksiyom	İsim	Koşul
T0	Kolmogorov	$\forall x \neq y: \exists U \in \tau$ ayıran
T1	Fréchet	$\forall x \neq y$: iki yönlü ayırım
T2	Hausdorff	$\forall x \neq y$: disjoint $U \ni x, V \ni y$
T2.5	Urysohn	disjoint kapanışlı komşuluklar
T3	Regüler	T1 + nokta/kapalı ayırma
T3.5	Tychonoff	T1 + sürekli fonksiyon ayırma
T4	Normal	T1 + disjoint kapalılar ayırma

Tablo 6.1: Ayrılma aksiyomları

Sıralama: $T4 \Rightarrow T3.5 \Rightarrow T3 \Rightarrow T2.5 \Rightarrow T2 \Rightarrow T1 \Rightarrow T0$.

6.2 Teoremler

Teorem 6.1 (Ayrılma Zinciri). $T4 \Rightarrow T3.5 \Rightarrow T3 \Rightarrow T2.5 \Rightarrow T2 \Rightarrow T1 \Rightarrow T0$. Tersini genel olarak doğru değildir.

Teorem 6.2 (Urysohn Fonksiyon Teoremi). X normal ise ve C, D disjoint kapalı kümeler ise, $f : X \rightarrow [0, 1]$ sürekli vardır: $f|_C \equiv 0, f|_D \equiv 1$.

Teorem 6.3 (Tietze Genişleme). X normal ise her kapalı $A \subseteq X$ üzerinde $f : A \rightarrow [a, b]$ sürekli tüm X 'e sürekli genişletilebilir.

Teorem 6.4 (Tychonoff Karakterizasyonu). $X, T3.5$ 'tir $\iff X$ bir küp $[0, 1]^I$ 'ye homeomorf gömülebilir.

Teorem 6.5 (Sonlu T1 $\iff$ Ayrık). Sonlu bir uzayda $T1 \iff$ ayrık topoloji.

İskelet. $T1 \Rightarrow$ her tekil küme kapalı $\Rightarrow$ her küme kapalı (sonlu birleşim) $\Rightarrow$ her küme açık. $\square$

6.3 Algoritmalar

Algorithm 8 Sonlu T0 Karar Prosedürü

```

1: for each pair  $(x, y)$  with  $x \neq y$  do
2:   if  $\nexists U \in \tau: (x \in U \wedge y \notin U) \text{ or } (y \in U \wedge x \notin U)$  then return false
3:   end if
4: end for return true

```

Karmaşıklık T0: $O(|X|^2 \cdot |\tau|)$. **Karmaşıklık T2:** $O(|X|^2 \cdot |\tau|^2)$.

6.4 pytop API

Listing 6.1: Ayrılma aksiyom importları

```

1 from pytop import (
2     is_t0, is_t1, is_t2, is_t2_5, is_t3, is_t4,
3     is_hausdorff, is_regular, is_normal, is_perfectly_normal,
4     separation_chain, analyze_separation,
5 )

```

Fonksiyon	İmza	Açıklama
is_t0 ...is_t4	(space)	Aksiyom kontrolü
separation_chain	(space)	Tüm zincir: dict[str, Result]
analyze_separation	(space, property = 'hausdorff')	Tek aksiyom sorgu

Tablo 6.2: Ayrılma fonksiyonları

6.5 Örnekler

6.5.1 Örnek 1: Sierpiński — T0 ✓, T1 ×

```

1 from pytop import sierpinski_space, is_t0, is_t1, is_t2
2 s = sierpinski_space()
3 print("T0:", is_t0(s).status)
4 print("T1:", is_t1(s).status)
5 print("T2:", is_t2(s).status)

```

Listing 6.2: Çıktı

```

1 T0: true
2 T1: false
3 T2: false

```

6.5.2 Örnek 2: Kosonlu $\mathbb{N}$ — $T_1 \checkmark$, $T_2 \times$

```

1 from pytop import naturals_cofinite, is_t1, is_t2
2 nc = naturals_cofinite()
3 print("T1:", is_t1(nc).status)
4 print("T2:", is_t2(nc).status)

```

Listing 6.3: Çıktı

```

1 T1: true
2 T2: false

```

6.5.3 Örnek 3: Ayrılma Zinciri — Ayrık

```

1 from pytop import discrete_topology, separation_chain
2 d = discrete_topology(1, 2, 3)
3 for prop, result in separation_chain(d).items():
4     print(f" {prop:20s}: {result.status}")

```

Listing 6.4: Çıktı (kısa)

```

1 t0 : true
2 t1 : true
3 hausdorff : true
4 t4 : true
5 perfectly_normal : true

```

6.6 Alıştırmalar

Kodlama

K1. `make_topology({1,2,3},{1},{2},{1,2})` için `separation_chain` çalıştırın.

K2. `finite_chain_space(3)` üzerinde en yüksek sağlanan aksiyomu bulun.

K3. `two_point_indiscrete_space()` üzerinde T_0 , T_1 , T_2 test edin.

Teori

T1. $T_2 \Rightarrow T_1 \Rightarrow T_0$ implicasyonlarını kanıtlayın.

T2. Sonlu uzayda $T_1 \iff$ ayrık topoloji olduğunu gösterin.

Bölüm 7

Kompaktlık

Kompaktlık, sonsuz yapılarda “sınırlılık” fikrinin topolojik soyutlamasıdır. Açık örtü tanımını temel alınarak incelenmekte; varyantlar ve lokal kompaktlık da ele alınmaktadır.

7.1 Konu

7.1.1 Kompaktlık Tanımı

Tanım 7.1 (Kompakt Uzak). (X, τ) topolojik uzayı **kompakt** ise: Her açık örtü $\{U_\alpha\}$ için sonlu alt-örtü vardır:

$$X \subseteq \bigcup_{\alpha} U_{\alpha} \implies \exists \text{ sonlu } I : X \subseteq \bigcup_{\alpha \in I} U_{\alpha}.$$

7.1.2 Kompaktlık Varyantları

Varyant	Tanım
Kompakt	Her açık örtünün sonlu alt-örtüsü
Sayılabılır kompakt	Her sayılabılır örtünün sonlu alt-örtüsü
Ardışıklı kompakt	Her dizi yakınsak alt-dizi içerir
Sözde kompakt	Her sürekli $f : X \rightarrow \mathbb{R}$ sınırlıdır
Lindelöf	Her açık örtünün sayılabılır alt-örtüsü
σ -kompakt	Sayılabılır kompakt kümelerin birleşimi
Lokal kompakt	Her noktanın kompakt komşuluğu var

Tablo 7.1: Kompaktlık varyantları

Sıralama: Kompakt $\Rightarrow$ Sayılabılır kompakt $\Rightarrow$ Ardışıklı kompakt $\Rightarrow$ Lindelöf.

7.2 Teoremler

Teorem 7.2 (Heine-Borel). $\mathbb{R}^n$ ’de kompakt $\Leftrightarrow$ kapalı ve sınırlı.

Teorem 7.3. Kompakt + Hausdorff $\Rightarrow$ Normal (T_4).

Teorem 7.4. Kompakt uzaydan Hausdorff uzaya sürekli bijeksiyon $\Rightarrow$ homeomorfizma.

Teorem 7.5 (Tychonoff). *Keyfi kompakt uzayların kartezyen çarpımı kompaktır.*

Teorem 7.6 (Alexandroff Tek-Nokta Kompaktifikasyonu). *Her lokal kompakt Hausdorff uzay X için $X^* = X \cup \{\infty\}$ kompakt Hausdorff'tur.*

7.3 Algoritmalar

7.3.1 Sonlu Uzayda Kompaktlık

Sonlu uzay her zaman kompaktır: her örtü zaten sonlu. Algoritma: $O(1)$.

7.3.2 analyze_compactness_variants — Tag Tabanlı Çıkarım

Algorithm 9 AnalyzeVariants(X, τ)

```

1: if  $X$  sonlu then
2:   tüm varyantlar  $\leftarrow$  True; return VariantProfile(...)
3: end if
4: if 'compact'  $\in$  tags then
5:   countably_compact, sequentially_compact, lindelof  $\leftarrow$  True
6: end if
7: if 'metric'  $\wedge$  'compact'  $\in$  tags then
8:   sequentially_compact  $\leftarrow$  True
9: end if
```

7.4 pytop API

```

1 from pytop import (
2     is_compact,
3     is_lindelof,
4     is_locally_compact,
5 )
6 from pytop.compactness_variants import (
7     is_countably_compact,
8     is_sequentially_compact,
9     analyze_compactness_variants,
10 )
```

analyze_compactness_variants(space) $\rightarrow$ Result; .value bir dict içerir.

7.5 Örnekler

Örnek 5.1 — Sonlu Uzaylar: Otomatik Kompakt

```

1 from pytop import sierpinski_space, discrete_topology,
   finite_chain_space, is_compact
2
```

```

3 s = sierpinski_space()
4 print("Sierpinski compact?", is_compact(s).status)
5 d = discrete_topology(1, 2, 3)
6 print("Discrete(3) compact?", is_compact(d).status)
7 c = finite_chain_space(3)
8 print("Chain(3) compact?", is_compact(c).status)

```

```

Sierpinski compact?      true
Chain(3) compact?       true
Discrete(3) compact?    true

```

Örnek 5.2 — Gerçek Doğru $\mathbb{R}$: Kompakt Değil

```

1 rl = real_line_metric()
2 print("compact?", is_compact(rl).status)
3 print("lindelof?", is_lindelof(rl).status)
4 print("locally_compact?", is_locally_compact(rl).status)

```

```

compact?                false
lindelof?               true
locally_compact?       conditional

```

Örnek 5.3 — analyze_compactness_variants: Sierpiński

```

1 r = analyze_compactness_variants(s)
2 for key, val in r.value.items():
3     if hasattr(val, 'status'):
4         print(f"  {key:<25}: {val.status}")

```

```

representation          : finite
countably_compact       : true
sequentially_compact    : true
pseudocompact           : true
lindelof                : true

```

7.6 Alıştırmalar

Kodlama

K1. `finite_chain_space(5)` ve `discrete_topology(1,2,3,4,5)` için `is_compact` karşılaştırın.

K2. `naturals_cofinite()` için `is_compact`, `is_lindelof`, `is_countably_compact` hesaplayın.

K3. `closed_unit_interval_metric()` ile `real_line_metric()` için `analyze_compactness_variants` çalıştırıp karşılaştırın.

Teori

- T1. Kompakt + sürekli $\Rightarrow$ görüntü kompakt olduğunu ispatlayın.
- T2. Heine-Borel teoreminin neden geçerli olduğunu: $[0, 1]$ neden kompakt, $(0, 1)$ neden kompakt değil?

Bölüm 8

Bağlantılılık

Bağlantılılık, bir topolojik uzayın iki ayrı açık parçaya bölünememesi özelliğidir. Yol-bağlantılılık daha güçlü bir kavramdır ve sürekli yolların varlığına dayanır.

8.1 Konu

8.1.1 Bağlantılılık Kavramları

Tanım 8.1 (Bağlantılı Uzay). (X, τ) uzayı **bağlantılı** ise: $X = U \cup V$, $U \cap V = \emptyset$, U, V açık $\Rightarrow U = \emptyset$ veya $V = \emptyset$.

Tanım 8.2 (Yol-Bağlantılı). $\forall x, y \in X: \exists$ sürekli $f : [0, 1] \rightarrow X$, $f(0) = x$, $f(1) = y$.

Kavram	Özellik
Bağlantılı	Clopen bölme yok
Yol-bağlantılı	Her iki nokta arasında sürekli yol var
Ark-bağlantılı	Yol-bağlantılı; yollar enjektif
Lokal bağlantılı	Her noktanın bağlantılı komşuluğu
Tamamen bağlantısız	Her bağlantılı alt küme tek noktadır

Tablo 8.1: Bağlantılılık hiyerarşisi

Sıralama: Ark-bağlantılı $\Rightarrow$ Yol-bağlantılı $\Rightarrow$ Bağlantılı.

8.1.2 Clopen Ayrılma

X bağlantısız $\Leftrightarrow$ boş olmayan trivial olmayan bir clopen $A \subsetneq X$ vardır.

8.2 Teoremler

Teorem 8.3. *Bağlantılı küme süreklilik altında bağlantılıdır: $f : X \rightarrow Y$ sürekli, X bağlantılı $\Rightarrow f(X)$ bağlantılı.*

Teorem 8.4. *Yol-bağlantılı $\Rightarrow$ bağlantılı. Tersisi genel olarak yanlış. (Karşı örnek: Topolojist sinüs eğrisi.)*

Teorem 8.5 (Ara Değer Teoremi). $f : X \rightarrow \mathbb{R}$ sürekli, X bağlantılı $\Rightarrow f(X)$ bir aralıktır.

Teorem 8.6. *Gerçek doğru $\mathbb{R}$ 'de bağlantılı alt kümeler tam olarak aralıklardır.*

8.3 Algoritmalar

8.3.1 Sonlu Bağlantılılık — $O(|\tau|^2)$

Algorithm 10 $\text{BagliMi}(X, \tau)$

```

1: for each non-empty  $A \subsetneq X$  do
2:   if  $A \in \tau$  and  $X \setminus A \in \tau$  then
3:     return False ▷ Clopen bölme bulundu
4:   end if
5: end for
6: return True

```

Karmaşıklık: $O(|\tau|^2)$ — clopen küme taraması.

8.4 pytop API

```

1 from pytop import (
2     is_connected,
3     is_path_connected,
4     is_locally_connected,
5     is_totally_disconnected,
6 )

```

8.5 Örnekler

Örnek 5.1 — Sierpiński: Bağlantılı

```

1 s = sierpinski_space()
2 print("connected?", is_connected(s).status)
3 print("path_connected?", is_path_connected(s).status)

```

```

connected?      true
path_connected? unknown

```

Örnek 5.2 — Ayırık Topoloji: Tamamen Bağlantısız

```

1 d = discrete_topology(1, 2, 3)
2 print("connected?", is_connected(d).status)
3 print("totally_disconn?", is_totally_disconnected(d).status)

```

```

connected?      false
totally_disconn? true

```

Örnek 5.3 — Gerçek Doğru ve $[0,1]$

```
1 r1 = real_line_metric()
2 ui = closed_unit_interval_metric()
3 print("R connected?", is_connected(r1).status)
4 print("[0,1] path_connected?", is_path_connected(ui).status)
```

```
R connected?          true
[0,1] path_connected? true
```

8.6 Alıştırmalar

Kodlama

- K1. `make_topology({1,2,3},{1},{2,3})` için `is_connected` hesaplayın.
- K2. `finite_chain_space(4)` zincirinin bağlantılı olup olmadığını kontrol edin.
- K3. İki noktalı ayrık ve indiscrete uzaylar üzerinde `is_connected`, `is_path_connected` karşılaştırın.

Teori

- T1. Bağlantılı + sürekli $\Rightarrow$ görüntü bağlantılı teoremini ispatlayın.
- T2. Yol-bağlantılı $\Rightarrow$ bağlantılı implicasyonunu ispatlayın.

Bölüm 9

Sayılabilirlik Aksiyomları

Sayılabilirlik aksiyomları, bir topolojik uzaydaki “bilgi”nin sayılabilir büyüklükte yapılarla tanımlanıp tanımlanamayacağını araştırır.

9.1 Konu

9.1.1 Sayılabilirlik Aksiyomları

Aksiyom	Tanım
Birinci sayılabilir	Her $x \in X$ için sayılabilir yerel baz
İkinci sayılabilir	Tüm topoloji için sayılabilir global baz
Ayrılabilir	Sayılabilir yoğun alt küme: $\overline{D} = X$
Lindelöf	Her açık örtünün sayılabilir alt-örtüsü

Tablo 9.1: Sayılabilirlik aksiyomları

Sıralamalar:

- 2^{nd} sayılabilir $\Rightarrow$ 1^{st} sayılabilir
- 2^{nd} sayılabilir $\Rightarrow$ ayrılabilir $\wedge$ Lindelöf
- Metrik $\Rightarrow$ 1^{st} sayılabilir
- Ayrılabilir metrik $\Rightarrow$ 2^{nd} sayılabilir

9.2 Teoremler

Teorem 9.1. 2^{nd} sayılabilir $\Rightarrow$ ayrılabilir $\wedge$ Lindelöf.

Teorem 9.2. Metrik uzay $\Rightarrow$ 1^{st} sayılabilir.

Kanıt İskeleti. Her x için $\{B(x, 1/n) : n \in \mathbb{N}\}$ sayılabilir yerel bazdır. □

Teorem 9.3. Ayrılabilir + metrik $\Rightarrow$ 2^{nd} sayılabilir.

Kanıt İskeleti. $D \subseteq X$ yoğun sayılabilir; $\{B(d, 1/n) : d \in D, n \in \mathbb{N}\}$ sayılabilir bazdır. □

Teorem 9.4 (Sorgenfrey Karşı Örneği). *Sorgenfrey doğrusu ayrılabilir ve Lindelöf'tür ama 2^{nd} sayılabilir değildir.*

9.3 Algoritmalar

9.3.1 Sonlu Uzayda Sayılabilirlik

Sonlu uzayda her aksiom trivially sağlanır. $O(1)$.

9.3.2 Sonsuz Uzayda: Tag-Tabanlı Çıkarım

pytop sembolik uzaylarda tag koridorlarından sayılabilirlik çıkarır:

- 'second_countable' $\in$ tags $\Rightarrow$ tüm aksiyomlar
- 'metric' $\wedge$ 'separable' $\Rightarrow$ 'second_countable'

9.4 pytop API

```
1 from pytop import (
2     is_first_countable,
3     is_second_countable,
4     is_separable,
5     is_lindelof,
6 )
```

9.5 Örnekler

Örnek 5.1 — Gerçek Doğru: Tüm 4 Aksiom

```
1 rl = real_line_metric()
2 print("1st countable?", is_first_countable(rl).status)
3 print("2nd countable?", is_second_countable(rl).status)
4 print("separable?", is_separable(rl).status)
5 print("lindehof?", is_lindelof(rl).status)
```

```
1st countable? true
2nd countable? true
separable?     true
lindehof?      true
```

Örnek 5.5 — Sorgenfrey Doğrusu: 2nd Countable $\times$

```
1 from pytop import sorgenfrey_line_like
2 sl = sorgenfrey_line_like()
3 print("1st countable?", is_first_countable(sl).status)
4 print("2nd countable?", is_second_countable(sl).status)
5 print("separable?", is_separable(sl).status)
```

```
1st countable?    true
2nd countable?    false
separable?        true
```

9.6 Alıştırmalar

Kodlama

- K1. `finite_chain_space(5)` üzerinde `is_first_countable` ve `is_second_countable` hesaplayın.
- K2. İki noktalı ayrık ve indiscrete uzaylar üzerinde `is_separable` karşılaştırın.
- K3. `integers_discrete()` için 2^{nd} sayılabilir mi?

Teori

- T1. 2^{nd} sayılabilir $\Rightarrow$ ayrılabilir olduğunu ispatlayın.
- T2. Sorgenfrey doğrusunun 2^{nd} sayılabilir olmadığını gösterin.

Bölüm 10

Süreklili Fonksiyonlar ve Homeomorfizmalar

Topolojik uzaylar arasındaki fonksiyonlarda süreklilik, açık kümelerin geri çekiminin açık olması koşuluna dayanır.

10.1 Konu

Tanım 10.1 (Süreklili Fonksiyon). $f : (X, \tau_X) \rightarrow (Y, \tau_Y)$ fonksiyonu **süreklili** ise:

$$\forall V \in \tau_Y : f^{-1}(V) \in \tau_X.$$

Eşdeğer koşullar:

- Her kapalı $F \subseteq Y$ için $f^{-1}(F)$ kapalıdır.
- Her $x \in X$ ve $f(x)$ 'in her komşuluğu W için $f^{-1}(W)$, x 'in komşuluğudur.

Tanım 10.2 (Homeomorfizma). $f : X \rightarrow Y$ **homeomorfizma** ise: biyektif, f süreklili, f^{-1} süreklili. $(X, \tau_X) \cong (Y, \tau_Y)$ ile gösterilir.

Tür	Koşul
Sabit fonksiyon	$f(x) = c$; her zaman süreklili
Özdeşlik	$f(x) = x$; her zaman süreklili
Bileşke	f, g süreklili $\Rightarrow g \circ f$ süreklili
Homeomorfizma	Biyektif; f ve f^{-1} süreklili

Tablo 10.1: Süreklilik hiyerarşisi

10.2 Teoremler

Teorem 10.3. Her sabit fonksiyon süreklidir.

Kanıt İskeleti. $f^{-1}(V) = X$ eğer $c \in V$, $\emptyset$ eğer $c \notin V$; her ikisi de açıktır. □

Teorem 10.4. $f : X \rightarrow Y$ ve $g : Y \rightarrow Z$ süreklili $\Rightarrow g \circ f$ süreklidir.

Teorem 10.5. $f : X \rightarrow Y$ sürekli, X kompakt $\Rightarrow f(X)$ kompaktır.

Teorem 10.6. $f : X \rightarrow Y$ sürekli, X bağlantılı $\Rightarrow f(X)$ bağlantılıdır.

Teorem 10.7. Kompakt X 'ten Hausdorff Y 'ye sürekli bijeksiyon $\Rightarrow$ homeomorfizma.

10.3 Algoritmalar

10.3.1 is_continuous_finite_map — $O(|\tau_Y| \cdot |X|)$

Algorithm 11 IsContinuous(X, τ_X, Y, τ_Y, f)

```

1: for each  $V \in \tau_Y$  do
2:   preimage  $\leftarrow \{x \in X : f(x) \in V\}$ 
3:   if preimage  $\notin \tau_X$  then
4:     return False
5:   end if
6: end for
7: return True

```

10.4 pytop API

```

1 from pytop import (
2     is_continuous_finite_map,
3     finite_homeomorphism_result,
4     sierpinski_space,
5     discrete_topology,
6     indiscrete_topology,
7 )

```

```

    is_continuous_finite_map(domain, domain_topology, codomain, codomain_topology,
mapping)  $\rightarrow$  bool
    finite_homeomorphism_result(left, right)  $\rightarrow$  Result

```

10.5 Örnekler

Örnek 5.2 — Ayırık Topoloji: Her Fonksiyon Sürekli

```

1 d = discrete_topology(0, 1)
2 d_pts = list(d.carrier)
3 d_topo = [set(u) for u in d.topology]
4
5 f_id = {0: 0, 1: 1}
6 f_swap = {0: 1, 1: 0}
7 print("ozdeslik:", is_continuous_finite_map(d_pts, d_topo, d_pts,
    d_topo, f_id))
8 print("swap:", is_continuous_finite_map(d_pts, d_topo, d_pts,
    d_topo, f_swap))

```

```
ozdeslik continuous: True
swap      continuous: True
```

Örnek 5.3 — Sierpiński $\rightarrow$ Ayırık: Sürekli Değil

```
1 s = sierpinski_space()
2 s_pts = list(s.carrier)
3 s_topo = [set(u) for u in s.topology]
4 print("id: S->D:", is_continuous_finite_map(s_pts, s_topo, d_pts,
5       d_topo, f_id))
6 print("id: D->S:", is_continuous_finite_map(d_pts, d_topo, s_pts,
7       s_topo, f_id))
```

```
id: Sierpinski -> Disc continuous: False
id: Disc -> Sierpinski continuous: True
```

Sierpiński $\tau = \{\emptyset, \{0, 1\}, \{1\}\}$: $f^{-1}(\{0\}) = \{0\}$ Sierpiński'de açık değil.

Örnek 5.5 — Homeomorfizma Kontrolü

```
1 d2a = discrete_topology(1, 2)
2 d2b = discrete_topology('a', 'b')
3 ind2 = indiscrete_topology(1, 2)
4 print("D(1,2) ~ D(a,b):", finite_homeomorphism_result(d2a, d2b).
5       status)
6 print("D(1,2) ~ S(0,1):", finite_homeomorphism_result(d2a, s).status)
7 print("D(1,2) ~ Ind(1,2):", finite_homeomorphism_result(d2a, ind2).
8       status)
```

```
D(1,2) ~ D(a,b): true
D(1,2) ~ S(0,1): false
D(1,2) ~ Ind(1,2): false
```

10.6 Alıştırımlar

Kodlama

- K1. Özel bir topolojide $f(0) = 0, f(1) = 0, f(2) = 1$ fonksiyonunun sürekli olup olmadığını kontrol edin.
- K2. Sierpiński uzayından kendisine giden dört fonksiyonu test edin.
- K3. `finite_homeomorphism_result` ile iki ayırık uzayın homeomorf olduğunu doğrulayın.

Teori

- T1. Her sabit fonksiyonun sürekli olduğunu ispatlayın.
- T2. $f \circ g$ bileşkesinin sürekli olduğunu ispatlayın.

Bölüm 11

Alt Uzay ve Çarpım Topolojisi

Alt uzay topolojisi bir uzayın alt kümesine miras kalan topolojiyi tanımlar. Çarpım topolojisi ise iki uzayı birleştirerek yeni bir uzay kurar.

11.1 Konu

Tanım 11.1 (Alt Uzay Topolojisi). (X, τ) uzayı ve $A \subseteq X$ verilsin. **Alt uzay topolojisi:**

$$\tau_A = \{U \cap A : U \in \tau\}.$$

Tanım 11.2 (Çarpım Topolojisi). (X, τ_X) ve (Y, τ_Y) verilsin. Çarpım topolojisi $X \times Y$ üzerinde $\{U \times V : U \in \tau_X, V \in \tau_Y\}$ bazından üretilir.

Projeksiyon $\pi_X : X \times Y \rightarrow X$ ve $\pi_Y : X \times Y \rightarrow Y$ süreklidir.

Özellik	Alt uzay	Çarpım
Hausdorff	✓	✓
Kompaktlık	Yalnız kapalı alt uzay	✓ (Tychonoff)
Bağlantılılık	Hayır (genel)	✓
T0/T1/T2	✓	✓

Tablo 11.1: Topolojik özelliklerin kalıtımı

11.2 Teoremler

Teorem 11.3. *Hausdorff uzayın her alt uzayı Hausdorff'tur.*

Teorem 11.4. *Kompakt uzayın kapalı alt uzayı kompaktır.*

Teorem 11.5. X ve Y bağlantılı $\Rightarrow X \times Y$ bağlantılıdır.

Kanıt İskeleti. $\{x_0\} \times Y$ ve $X \times \{y_0\}$ bağlantılı dilimler; kesişimleri üzerinden birleşerek tüm çarpımı kapsar. $\square$

Teorem 11.6 (Tychonoff, $n = 2$). X ve Y kompakt $\Rightarrow X \times Y$ kompaktır.

Teorem 11.7 (Evrensel Özellik — Çarpım). $f : Z \rightarrow X \times Y$ sürekli $\Leftrightarrow \pi_X \circ f$ ve $\pi_Y \circ f$ süreklidir.


```
carrier: (1, 2)
topology: [[], [1], [2], [1, 2]]
```

Örnek 5.4 — Çarpım: $D(0, 1) \times D(0, 1)$

```
1 d2 = discrete_topology(0, 1)
2 prod_dd = binary_product(d2, d2)
3 print("carrier:", sorted(prod_dd.carrier))
4 print("topology size:", len(list(prod_dd.topology)))
5 print("compact:", is_compact(prod_dd).status)
6 print("connected:", is_connected(prod_dd).status)
```

```
carrier: [(0, 0), (0, 1), (1, 0), (1, 1)]
topology size: 16
compact: true
connected: false
```

Örnek 5.5 — Çarpım: Sierpiński $\times$ Sierpiński

```
1 ss = binary_product(s, s)
2 print("topology size:", len(list(ss.topology)))
3 print("compact:", is_compact(ss).status)
4 print("connected:", is_connected(ss).status)
```

```
topology size: 6
compact: true
connected: true
```

11.6 Alıştırmalar

Kodlama

- K1. Sierpiński uzayının $\{0\}$ ve $\{1\}$ alt uzaylarını oluşturun ve hangi topolojiyi taşıdığını belirleyin.
- K2. 4-noktalı özel bir uzayda $\{2, 3\}$ alt uzayını bulun.
- K3. `binary_product(sierpinski_space(), discrete_topology(0,1))` için `is_compact` ve `is_connected` kontrol edin.

Teori

- T1. Alt uzay topolojisinde dahil etme $i : A \rightarrow X$ 'in sürekli olduğunu ispatlayın.
- T2. X ve Y bağlantılı $\Rightarrow X \times Y$ bağlantılı olduğunu ispatlayın.

Bölüm 12

Bölüm Topolojisi

Bölüm topolojisi, bir topolojik uzayın noktalarını denklik sınıflarına göre *yapıştırarak* yeni bir uzay kurar. Alt uzay ve çarpım topolojisiyle birlikte temel üç inşaat yönteminden birini oluşturur.

12.1 Konu

Tanım 12.1 (Bölüm Kümesi). (X, τ) topolojik uzayı ve X üzerinde bir denklik bağıntısı $\sim$ verilsin. Denklik sınıfı $[x] = \{y \in X : y \sim x\}$. **Bölüm kümesi:** $X/\sim = \{[x] : x \in X\}$.

Tanım 12.2 (Bölüm Topolojisi). Bölüm haritası $q : X \rightarrow X/\sim$, $q(x) = [x]$. **Bölüm topolojisi** $\tau_{X/\sim}$ üzerinde:

$$U \subseteq X/\sim \text{ açıktır} \iff q^{-1}(U) \in \tau.$$

Bu, q 'yu sürekli kılan en ince topolojidir.

Özellik	Bölüm uzayı için durum
Kompaktlık	X kompakt $\Rightarrow X/\sim$ kompakt
Bağlantılılık	X bağlantılı $\Rightarrow X/\sim$ bağlantılı
Hausdorff	Genel olarak korunmaz
T1	$\sim$ kapalı bağıntı ise korunur

Tablo 12.1: Topolojik özelliklerin bölüm altında korunumu

12.2 Teoremler

Teorem 12.3 (Bölüm haritası sürekliliği). $q : X \rightarrow X/\sim$ her zaman süreklidir.

Teorem 12.4 (Evrensel Özellik). $g : X/\sim \rightarrow Z$ olsun. g süreklidir $\iff g \circ q : X \rightarrow Z$ süreklidir.

Teorem 12.5 (Kompaktlık Korunumu). X kompakt $\Rightarrow X/\sim$ kompaktır.

Kanıt İskeleti. q sürekli ve surjektif; kompakt kümenin sürekli görüntüsü kompaktır. $\square$

Teorem 12.6 (Bağlantılılık Korunumu). X bağlantılı $\Rightarrow X/\sim$ bağlantılıdır.

Teorem 12.7 (Hausdorff Kriteri). $X/\sim$ Hausdorff $\iff \sim$ grafiği $X \times X$ 'te kapalıdır.

12.3 Algoritmalar

12.3.1 quotient_set — $O(|X|^2 \cdot \alpha(|X|))$

Algorithm 14 QuotientSet($X, \sim$)

```

1: Her  $x$  için sınıf  $\leftarrow \{x\}$  (union-find başlat)
2: for each  $(x, y) \in \sim$  do
3:   Union( $x, y$ )
4: end for
5: return kök temsilcilere göre sınıfları tuple olarak

```

12.3.2 finite_quotient_contract — $O(|X| + |P|)$

Algorithm 15 FiniteQuotientContract(X, P)

```

1: //  $P$ :  $X$ 'in örtüşmeyen bölüntüsü
2: Doğrula:  $P$  her  $x$ 'i kapsar ve bloklar ayrık
3: return FiniteConstructionContract(status =true, carrier_size = $|X|$ ,
   block_count = $|P|$ )

```

12.4 pytop API

```

1 from pytop import (
2     quotient_set,           # denklik bagintisinden bolum kumesi
3     finite_quotient_contract, # boluntuden infaat sozlesmesi
4     finite_quotient_summary, # kisa ozet dizesi
5     make_quotient_map,      # QuotientMap nesnesi
6     is_quotient_map,        # Result: harita bolum haritasi mi?
7 )

```

```

quotient_set(carrier, relation) → tuple[frozenset, ...]
finite_quotient_contract(carrier, partition) → FiniteConstructionContract
finite_quotient_summary(carrier, partition) → str
make_quotient_map(domain, codomain) → QuotientMap
is_quotient_map(map_obj) → Result

```

12.5 Örnekler

Örnek 5.1 — İki Noktayı Özdeşleştirme

```

1 d4 = discrete_topology(0, 1, 2, 3)
2 partition_1 = [[0, 1], [2], [3]]
3 qs1 = quotient_set(
4     [0, 1, 2, 3],

```

```

5      [(0,0),(1,1),(2,2),(3,3),(0,1),(1,0)]
6  )
7  print("Bolum kumesi:", qs1)
8  contract1 = finite_quotient_contract([0, 1, 2, 3], partition_1)
9  print("Blok sayisi:", contract1.block_count)
10 print("Durum:", contract1.status)

```

Bolum kumesi: (frozenset({2}), frozenset({3}), frozenset({0, 1}))
 Blok sayisi: 3
 Durum: true

Örnek 5.2 — Tüm Noktaları Tek Noktaya Çökertme

```

1 carrier = [0, 1, 2, 3]
2 qs2 = quotient_set(
3     carrier,
4     [(i, j) for i in carrier for j in carrier]
5 )
6 print("Tek blok:", qs2)
7 contract2 = finite_quotient_contract(carrier, [carrier])
8 print("Blok sayisi:", contract2.block_count)
9 r2 = contract2.to_result()
10 print("Metadata:", r2.metadata['block_sizes'])

```

Tek blok: (frozenset({0, 1, 2, 3})),
 Blok sayisi: 1
 Metadata: [4]

Örnek 5.3 — Özel Topolojide Bölüm Sözleşmesi

```

1 X5 = [1, 2, 3, 4, 5]
2 tau5 = [set(), {1,2}, {3,4}, {1,2,3,4}, {1,2,3,4,5}]
3 fts5 = make_topology(X5, *tau5)
4 partition5 = [[1, 2], [3, 4], [5]]
5 contract5 = finite_quotient_contract(X5, partition5)
6 r5 = contract5.to_result()
7 print("Status:", r5.status)
8 print("Blokler:", r5.metadata['block_count'])
9 print("Blok boyutlari:", r5.metadata['block_sizes'])

```

Status: true
 Bloklar: 3
 Blok boyutlari: [2, 2, 1]

Örnek 5.4 — Bölüm Haritası Nesnesi

```

1 d3 = discrete_topology(0, 1, 2)
2 d2 = discrete_topology('a', 'b')
3 qmap = make_quotient_map(d3, d2)
4 print("Etiketler:", sorted(qmap.tags))
5 r_qmap = is_quotient_map(qmap)
6 print("is_quotient_map status:", r_qmap.status)
7 print("Justification:", r_qmap.justification[0])

```

Etiketler: ['continuous', 'quotient', 'surjective']
 is_quotient_map status: true
 Justification: Map tag 'quotient' is explicitly present.

12.6 Alıştırmalar

Kodlama

- K1. `discrete_topology(0,1,2,3,4)` üzerinde `[[0,4],[1,3],[2]]` bölüntüsünü kullanarak `finite_quotient_contract` çağırın; blok sayısını ve boyutlarını yazdırın.
- K2. `sierpinski_space()` üzerinde trivial bölüntü ile `finite_quotient_summary` çağırın ve çıktıyı yorumlayın.
- K3. `make_topology([1,2,3,4], set(), {1,2}, {3,4}, {1,2,3,4})` üzerinde `[[1,2],[3,4]]` ve `[[1],[2],[3],[4]]` bölüntülerini karşılaştırın.

Teori

- T1. $q : X \rightarrow X/\sim$ bölüm haritasının $\tau_{X/\sim}$ tanımı gereği her zaman sürekli olduğunu ispatlayın.
- T2. X bağlantılı $\Rightarrow X/\sim$ bağlantılı olduğunu ispatlayın. (q sürekli ve surjektif; bağlantılı kümenin sürekli görüntüsü bağlantılıdır.)

Bölüm 13

Başlangıç ve Son Topoloji

Başlangıç topolojisi (initial topology), verilen haritaları sürekli kılan en kaba (coarsest) topolojidir. **Son topoloji** (final topology), verilen haritaları sürekli kılan en ince (finest) topolojidir. Altuzay ve çarpım topolojileri başlangıç topolojisinin; bölüm topolojisi ise son topolojisinin özel halleridir.

13.1 Başlangıç Topolojisi

X kümesi ve haritalar ailesi $\{f_\alpha : X \rightarrow (Y_\alpha, \tau_\alpha)\}_{\alpha \in A}$ verilsin.

Tanım 13.1 (Başlangıç Topolojisi). **Başlangıç topolojisi** τ_{ini} , her f_α 'yı $(X, \tau_{\text{ini}}) \rightarrow (Y_\alpha, \tau_\alpha)$ sürekli kılan topolojilerin kesişimidir. Alt-baz:

$$\mathcal{S} = \{f_\alpha^{-1}(U) \mid U \in \tau_\alpha, \alpha \in A\}, \quad \tau_{\text{ini}} = \langle \mathcal{S} \rangle.$$

Teorem 13.2 (Varlık ve Teklik). τ_{ini} *tekil ve vardır; her f_α 'yı sürekli kılan tüm topolojilerin en kaba olanıdır.*

Kanıt İskeleti. $\mathcal{S}$ 'nin ürettiği topoloji $\langle \mathcal{S} \rangle$ her f_α 'yı sürekli kılar (çünkü $f_\alpha^{-1}(U) \in \langle \mathcal{S} \rangle$ her $U \in \tau_\alpha$ için). Eğer τ' de her f_α 'yı sürekli kılıyorsa, $\mathcal{S} \subseteq \tau'$ olduğundan $\langle \mathcal{S} \rangle \subseteq \tau'$; yani τ_{ini} en kaba. $\square$

Teorem 13.3 (Evrensel Özellik). $h : Z \rightarrow (X, \tau_{\text{ini}})$ *sürekli ancak ve ancak $f_\alpha \circ h$ her α için sürekli olduğunda.*

Teorem 13.4 (Özel Haller). 1. *Altuzay topolojisi, ekleme haritası $i : A \hookrightarrow X$ 'in başlangıç topolojisidir.*

2. *Çarpım topolojisi, projeksiyon haritaları $\{\pi_\alpha\}$ 'nın başlangıç topolojisidir.*

13.2 Algoritmalar

13.2.1 Başlangıç Topolojisi İnşası

Karmaşıklık. Ön-görüntü hesabı $O(|\tau_\alpha| \cdot |X|)$; topoloji üretimi $O(|\mathcal{S}| \cdot 2^{|X|})$ en kötü durumda.

Algorithm 16 Başlangıç Topolojisi (Sonlu Durum)**Require:** Sonlu taşıyıcı X , haritalar $\{(f_\alpha, \tau_\alpha)\}$ **Ensure:** τ_{ini} — en kaba süreklilik topolojisi

```

1:  $\mathcal{S} \leftarrow \emptyset$ 
2: for her  $f_\alpha$  do
3:   for her  $U \in \tau_\alpha$  do
4:      $\mathcal{S} \leftarrow \mathcal{S} \cup \{f_\alpha^{-1}(U)\}$ 
5:     //  $f_\alpha^{-1}(U) = \{x \in X \mid f_\alpha(x) \in U\}$ 
6:   end for
7: end for
8: return  $\langle \mathcal{S} \rangle$  // alt-bazdan topoloji üret

```

Algorithm 17 Son Topoloji (Sonlu Durum)**Require:** Sonlu Y , kaynak uzaylar $\{(X_i, \sigma_i)\}$, haritalar $\{g_i : X_i \rightarrow Y\}$ **Ensure:** τ_{fin} — en ince süreklilik topolojisi

```

1:  $\tau_{\text{fin}} \leftarrow \emptyset$ 
2: for her  $V \subseteq Y$  do
3:   if  $g_i^{-1}(V) \in \sigma_i$  her  $i$  için then
4:      $\tau_{\text{fin}} \leftarrow \tau_{\text{fin}} \cup \{V\}$ 
5:   end if
6: end for
7: return  $\tau_{\text{fin}}$ 

```

13.2.2 Son Topoloji İnşasıKarmaşıklık. $O(2^{|Y|} \cdot \sum_i |\sigma_i|)$.**13.3 pytop API**

Listing 13.1: Başlangıç Topolojisi API

```

1 from pytop.finite_map_analysis import FiniteMap
2 from pytop import (
3     initial_topology_from_maps,
4     generic_quotient_space_from_map,
5 )
6
7 # FiniteMap dogrudan veri sinifi olarak kullanilir (make_function
8   degil)
9 f = FiniteMap(domain=domain_space, codomain=codomain_space,
10               name="f", mapping={...})
11
12 # Baslangic topolojisi
13 tau_ini = initial_topology_from_maps(carrier, [f1, f2, ...])
14
15 # Son topoloji (bolum ozel hali)
16 quot = generic_quotient_space_from_map(q_map)

```

Önemli: `pytop.quotient_space_from_map` sembolik motoru kullanır; sonlu-duyarlı sürüm `generic_quotient_space_from_map`'tir.

13.4 Örnekler

13.4.1 Tek Harita ile Başlangıç Topolojisi

```

1 X_carrier = [0, 1, 2, 3]
2 Y_d = discrete_topology(0, 1)
3 X_ph = make_topology(X_carrier, set(), set(X_carrier))
4
5 f = FiniteMap(domain=X_ph, codomain=Y_d, name="f",
6             mapping={0: 0, 1: 0, 2: 1, 3: 1})
7 tau_f = initial_topology_from_maps(X_carrier, [f])
8
9 print("Baslangic topolojisi:")
10 for u in sorted([sorted(u) for u in tau_f.topology], key=lambda x: (
11     len(x), x)):
12     print("  ", u if u else "empty")
13 print("Eleman sayisi:", len(tau_f.topology))

```

```

Baslangic topolojisi:
  empty
  [0, 1]
  [2, 3]
  [0, 1, 2, 3]
Eleman sayisi: 4

```

$f^{-1}(\{0\}) = \{0, 1\}$, $f^{-1}(\{1\}) = \{2, 3\}$ — alt-baz $\{\{0, 1\}, \{2, 3\}\}$, üretilen topoloji 4 elemanlı.

13.4.2 İki Harita — Topoloji İncelir

```

1 S_sier = sierpinski_space()
2 g = FiniteMap(domain=X_ph, codomain=S_sier, name="g",
3             mapping={0: 0, 1: 1, 2: 1, 3: 1})
4 tau_fg = initial_topology_from_maps(X_carrier, [f, g])
5
6 print("f tek basina:", len(tau_f.topology), "eleman | f+g:", len(
7     tau_fg.topology), "eleman")

```

```
f tek basina: 4 eleman | f+g: 6 eleman
```

$g^{-1}(\{1\}) = \{1, 2, 3\}$ yeni bir alt-baz elemanı ekler; kesişim $\{0, 1\} \cap \{1, 2, 3\} = \{1\}$ yeni bir açık küme üretir.

13.4.3 Altuzay = Başlangıç

```

1 fts_X = make_topology([0, 1, 2, 3], set(), {0, 1}, {2, 3}, {0, 1, 2,
2   3})
3
4 A_dom = discrete_topology(1, 2)
5 inc   = FiniteMap(domain=A_dom, codomain=fts_X, name="i", mapping
6   ={1: 1, 2: 2})
7
8 init_A = initial_topology_from_maps([1, 2], [inc])
9
10 sub_tau = sorted([sorted(u) for u in sub_A.topology], key=lambda x:
11   (len(x), x))
12 init_tau = sorted([sorted(u) for u in init_A.topology], key=lambda x:
13   (len(x), x))
14 print("Esit mi:", sub_tau == init_tau)

```

```
Esit mi: True
```

13.4.4 Çarpım = Başlangıç

```

1 d2 = discrete_topology(0, 1)
2 prod = binary_product(d2, d2)
3
4 pi_x = FiniteMap(domain=prod, codomain=d2, name="pi_x",
5   mapping={0,0):0, (0,1):0, (1,0):1, (1,1):1})
6 pi_y = FiniteMap(domain=prod, codomain=d2, name="pi_y",
7   mapping={0,0):0, (0,1):1, (1,0):0, (1,1):1})
8
9 init_prod = initial_topology_from_maps(list(prod.carrier), [pi_x,
10   pi_y])
11 prod_fs = {frozenset(u) for u in prod.topology}
12 init_fs = {frozenset(u) for u in init_prod.topology}
13 print("Carpim == Baslangic:", prod_fs == init_fs)

```

```
Carpim == Baslangic: True
```

13.4.5 Son Topoloji — Manuel Hesap

```

1 X1 = make_topology([0, 1, 2], set(), {0, 1}, {0, 1, 2})
2 Y_car = [0, 1]
3 g1_map = {0: 0, 1: 0, 2: 1}
4 tau_X1 = {frozenset(u) for u in X1.topology}
5
6 open_Y = []
7 for mask in range(1 << len(Y_car)):
8     V = frozenset(Y_car[i] for i in range(len(Y_car)) if mask
9       & (1 << i))
10    preimage = frozenset(x for x in X1.carrier if g1_map[x] in V)
11    if preimage in tau_X1:

```



```

11     open_Y.append(set(V))
12
13 final_Y = make_topology(Y_car, *open_Y)
14 final_tau = sorted([sorted(u) for u in final_Y.topology], key=lambda
15     x: (len(x), x))
16 print("Son topoloji:", final_tau)

```

```
Son topoloji: [[], [0], [0, 1]]
```

13.4.6 Bölüm = Son Topoloji

```

1 Y_ind = indiscrete_topology(0, 1)
2 q = FiniteMap(domain=X1, codomain=Y_ind, name="q",
3     mapping={0: 0, 1: 0, 2: 1})
4 quot_Y = generic_quotient_space_from_map(q)
5 quot_tau = sorted([sorted(u) for u in quot_Y.topology], key=lambda x:
6     (len(x), x))
7 print("Bolum topolojisi:", quot_tau)
8 print("Son topoloji ile esit:", quot_tau == final_tau)

```

```
Bolum topolojisi: [[], [0], [0, 1]]
Son topoloji ile esit: True
```

13.5 Alıştırmalar

1. $f : \{a, b, c, d\} \rightarrow \text{discrete}(\{0, 1\})$, $f(a) = f(b) = 0$, $f(c) = f(d) = 1$ için `initial_topology_from_` kullanın ve sonucu `finite_subspace` ile karşılaştırın.
2. Çarpım özel halinde yalnızca π_x kullanıldığında başlangıç topolojisi ne olur? `Discrete`'den daha kaba mı?
3. $X = \{0, 1, 2, 3, 4\}$, $\sigma_X = \text{discrete}$, $g : X \rightarrow \{0, 1, 2\}$ haritası için son topolojiyi manuel hesaplayın.
4. (Teori) $h : Z \rightarrow (X, \tau_{\text{ini}})$ sürekliliğinin $f_\alpha \circ h$ sürekliliğine denk olduğunu ispatlayın.
5. (Teori) τ_{fin} 'den daha ince hiçbir topoloji her g_i 'yi sürekli kılamaz; bunu tanımdaki en-ince koşulundan türetin.

Kısım III

Metrik Uzaylar

Bölüm 14

Metrik Uzaylar

Metrik uzaylar, mesafe fonksiyonuna dayanan topolojik yapılardır. Her metrik uzay doğal olarak bir topolojik uzay oluşturur; bu yapı “indüklenen topoloji” olarak adlandırılır.

14.1 Konu

14.1.1 Metrik Aksiyomları

Tanım 14.1 (Metrik Uzay). M kümesi ve $d : M \times M \rightarrow [0, \infty)$ verilsin. (M, d) bir **metrik uzay**, d ise **metrik** ise:

(M1) **Özdeşlik**: $d(x, y) = 0 \Leftrightarrow x = y$

(M2) **Simetri**: $d(x, y) = d(y, x)$

(M3) **Üçgen**: $d(x, z) \leq d(x, y) + d(y, z)$

14.1.2 Temel Kavramlar

- **Açık top**: $B(x, r) = \{y \in M : d(x, y) < r\}$
- **Kapalı top**: $\bar{B}(x, r) = \{y \in M : d(x, y) \leq r\}$
- **İndüklenen topoloji**: $\tau_d = \{U : \forall x \in U, \exists \varepsilon > 0, B(x, \varepsilon) \subseteq U\}$
- **Çap**: $\text{diam}(A) = \sup\{d(x, y) : x, y \in A\}$

14.1.3 Standart Metrikler

Metrik	Tanım	Uzay
Öklid	$d(x, y) = x - y $	$\mathbb{R}$
Ayrık	$d(x, y) = 0$ veya 1	Herhangi X
ℓ^∞ (max)	$\max_i d_i(x_i, y_i)$	Çarpım uzayı
Sınırlı (capped)	$d'(x, y) = \min(d(x, y), 1)$	Eşdeğer topoloji

Tablo 14.1: Standart metrikler

14.2 Teoremler

Teorem 14.2. Her metrik uzay Hausdorff (T_2) ve 1^{st} sayılabiliridir.

Kanıt İskeleti. $B(x, d(x, y)/2)$ ve $B(y, d(x, y)/2)$ ayrık komşuluklardır; $\{B(x, 1/n)\}$ yerel bazdır. $\square$

Teorem 14.3 (Sınırlı Metrik Eşdeğerliği). $d'(x, y) = \min(d(x, y), 1)$ ile d aynı topolojiyi indükler.

Teorem 14.4. $\mathbb{R}^n$ üzerindeki $\max$, $\sum$ ve Öklid metrikleri topolojik olarak eşdeğerdir.

14.3 Algoritmalar

14.3.1 validate_metric — $O(|X|^3)$

Algorithm 18 ValidateMetric(X, d)

```

1: for each  $x \in X$  do
2:   if  $d(x, x) \neq 0$  then return False
3:   end if
4:   for each  $y \neq x$  do
5:     if  $d(x, y) = 0$  then return False
6:     end if
7:   end for
8: end for
9: for each  $x, y \in X$  do
10:  if  $d(x, y) \neq d(y, x)$  then return False
11:  end if
12: end for
13: for each  $x, y, z \in X$  do
14:  if  $d(x, z) > d(x, y) + d(y, z)$  then return False
15:  end if
16: end for
17: return True

```

Karmaşıklık: $O(|X|^3)$ — üçgen eşitsizliği dominates.

14.3.2 open_ball — $O(|X|)$

Algorithm 19 OpenBall(x, r, X, d)

```

1: return  $\{y \in X : d(x, y) < r\}$ 

```

14.4 pytop API

```

1 from pytop.metric_spaces import (
2     FiniteMetricSpace,
3     SymbolicMetricSpace,
4     validate_metric,
5 )
6 from pytop import (
7     open_ball,
8     closed_ball,
9     distance_to_subset,
10    diameter_of_subset,
11    is_bounded_subset,
12    capped_metric,
13 )

```

`FiniteMetricSpace(carrier =tuple, distance =dict)` — mesafe sözlüğü (a,b): float.

`open_ball(fms, point, radius) → set` döner (Result değil).

`capped_metric(distance_fn, cap =1.0) → Callable` döner.

14.5 Örnekler

Örnek 5.1 — Ayırık Metrik Uzay

```

1 points = [1, 2, 3, 4]
2 dist_discrete = {(a, b): (0 if a == b else 1)
3                     for a in points for b in points}
4 fms = FiniteMetricSpace(carrier=tuple(points), distance=dist_discrete
5 )
6 print("carrier:", fms.carrier)
7 print("d(1,2):", fms.distance[(1,2)])

```

carrier: (1, 2, 3, 4)

d(1,2): 1

Örnek 5.3 — open_ball

```

1 print("B(1, 0.5) =", open_ball(fms, 1, 0.5))
2 print("B(1, 1.5) =", open_ball(fms, 1, 1.5))

```

B(1, 0.5) = {1}

B(1, 1.5) = {1, 2, 3, 4}

Örnek 5.6 — capped_metric

```
1 cap_fn = capped_metric(fms_line.distance, cap=1.0)
2 print("d(0,3) capped:", cap_fn(0, 3))
3 print("d(0,1) capped:", cap_fn(0, 1))
```

d(0,3) capped: 1.0

d(0,1) capped: 1.0

14.6 Alıştırmalar

Kodlama

- K1. $\{A, B, C, D\}$ üzerinde bir metrik tanımlayın ve `validate_metric` ile doğrulayın. Üçgen eşitsizliğini ihlal eden bir örnek de deneyin.
- K2. Öklid metrikli $\{0, \dots, 9\}$ üzerinde $B(5, 3.0)$ ve $\bar{B}(5, 3.0)$ hesaplayın.
- K3. `normalized_metric` kullanarak bir metriği $[0, 1]$ 'e normalleştirin.

Teori

- T1. Her metrik uzayın Hausdorff olduğunu ispatlayın.
- T2. $d' = \min(d, 1)$ ve d aynı topolojiyi indüklediğini gösterin.

Bölüm 15

Metrik Tamlık ve Kompaktlık

Metrik tamlık (completeness), bir metrik uzayda Cauchy dizilerinin yakınsayıp yakınsamadığını araştıran temel bir kavramdır.

15.1 Konu

15.1.1 Cauchy Dizisi ve Tamlık

Tanım 15.1 (Cauchy Dizisi). (M, d) metrik uzayında $\{x_n\}$ dizisi **Cauchy** ise:

$$\forall \varepsilon > 0, \exists N \in \mathbb{N} : m, n \geq N \Rightarrow d(x_m, x_n) < \varepsilon.$$

Tanım 15.2 (Tam Metrik Uzay). (M, d) **tam** ise: Her Cauchy dizisi M 'de bir limite yakınsır.

15.1.2 Tamamen Sınırlılık

$A \subseteq M$ **tamamen sınırlı** ise: her $\varepsilon > 0$ için A 'nın sonlu bir ε -net kümesi vardır.

$$S = \{s_1, \dots, s_n\} \subseteq M, \quad \forall x \in A \exists i : d(x, s_i) < \varepsilon.$$

15.1.3 Metrik Kompaktlık Karakterizasyonu

$$(M, d) \text{ tam} \wedge \text{tamamen sınırlı} \iff (M, d) \text{ kompakt}.$$

Uzay	Tam?	Tam. sınırlı?	Kompakt?
$\mathbb{R}$	✓	×	×
$[0, 1]$	✓	✓	✓
$(0, 1)$	×	✓	×
$\mathbb{Q}$	×	×	×
Sonlu metrik	✓	✓	✓

Tablo 15.1: Tamlık ve kompaktlık örnekleri

15.2 Teoremler

Teorem 15.3 (Heine-Borel Metrik Genelleştirmesi). *Metrik uzayda kompaktlık $\Leftrightarrow$ tamlık $\wedge$ tamamen sınırlılık.*

Teorem 15.4 (Banach Sabit-Nokta Teoremi). *(M, d) tam metrik uzay; $T : M \rightarrow M$ büzülme ($K < 1$) $\Rightarrow T$ 'nin eşsiz sabit noktası x^* vardır: $T(x^*) = x^*$.*

Teorem 15.5 (Baire Kategorisi Teoremi). *Tam metrik uzay 1. kategoriden değildir.*

15.3 Algoritmalar

15.3.1 Sonlu Metrikte is__complete ve is__totally__bounded

Sonlu metrik uzayda her Cauchy dizisi eninde sonunda sabit: trivially tam. Taşıyıcı kendisi ε -net: trivially tamamen sınırlı. Her ikisi: $O(1)$.

15.3.2 metric__compactness__check

Algorithm 20 MetricCompactnessCheck(M, d)

```

1: complete  $\leftarrow$  is__complete( $M, d$ )
2: totally_bounded  $\leftarrow$  is__totally__bounded( $M, d$ )
3: return complete  $\wedge$  totally_bounded

```

15.4 pytop API

```

1 from pytop.metric_completeness import (
2     is_complete,
3     is_totally_bounded,
4     metric_compactness_check,
5     analyze_metric_completeness,
6 )

```

15.5 Örnekler

Örnek 5.1 — Sonlu Metrik: Tam ve Kompakt

```

1 points = ['A', 'B', 'C', 'D']
2 dist = {(a, b): (0 if a == b else 1)
3         for a in points for b in points}
4 fms = FiniteMetricSpace(carrier=tuple(points), distance=dist)
5 print("is_complete:", is_complete(fms).status)
6 print("is_totally_bounded:", is_totally_bounded(fms).status)
7 mc = metric_compactness_check(fms)
8 print("metric_compactness:", mc.status, mc.value)

```



```
is_complete: true
is_totally_bounded: true
metric_compactness: true True
```

Örnek 5.4 — analyze_metric_completeness

```
1 r = analyze_metric_completeness(fms)
2 print("status:", r.status)
3 print("value:", r.value)
```

```
status: true
value: {'is_complete': True, 'is_totally_bounded': True, 'metric_compact': True}
```

15.6 Alıştırmalar

Kodlama

- K1. Rasyonel sayılar $\mathbb{Q}$ (sembolik) için `is_complete` sonucunu kontrol edin.
- K2. Kendi 4-noktalı metrik uzayınızı oluşturun ve `metric_compactness_check` uygulayın.
- K3. `analyze_metric_completeness` çalıştırın ve `value` sözlüğünü inceleyin.

Teori

- T1. Banach sabit-nokta teoremini sözlü olarak açıklayın ve bir uygulama verin.
- T2. Kapalı $[0, 1]$ 'in tam, açık $(0, 1)$ 'in tam olmadığını gösterin. (Hint: $1/n$ dizisi Cauchy ama $0 \notin (0, 1)$.)

Bölüm 16

Metrik Fonksiyonlar ve Sözleşmeler

Bu bölümde metrik uzaylar arasındaki fonksiyon türleri (izometri, Lipschitz, sürekli) ve bunların sınıflandırılması incelenmektedir.

16.1 Konu

16.1.1 Metrik Fonksiyon Sınıfları

$f : (M_1, d_1) \rightarrow (M_2, d_2)$ fonksiyonu:

Sınıf	Koşul
Genişlemez (non-expansive)	$d_2(f(x), f(y)) \leq d_1(x, y)$
Lipschitz	$\exists K > 0 : d_2(f(x), f(y)) \leq K \cdot d_1(x, y)$
Üniform sürekli	$\forall \varepsilon > 0 \exists \delta > 0 : d_1(x, y) < \delta \Rightarrow d_2(f(x), f(y)) < \varepsilon$
Sürekli	Her noktada ε - δ sürekliliği
İzometri	$d_2(f(x), f(y)) = d_1(x, y)$
Benzerlik	$\exists c > 0 : d_2(f(x), f(y)) = c \cdot d_1(x, y)$
Homeomorfizma	Bijektif sürekli; ters de sürekli

Tablo 16.1: Metrik fonksiyon sınıfları

Sıralama: İzometri $\Rightarrow$ Genişlemez $\Rightarrow$ Lipschitz $\Rightarrow$ Üniform sürekli $\Rightarrow$ Sürekli.

16.1.2 Lipschitz Sabiti

$$K = \sup_{x \neq y} \frac{d_2(f(x), f(y))}{d_1(x, y)}.$$

İzometri: $K = 1$ ve eşitlik; benzerlik: $K = c$ sabit.

16.2 Teoremler

Teorem 16.1. *İzometri $\Rightarrow$ homeomorfizma.*

Teorem 16.2. *Benzerlik, $c = 1 \Rightarrow$ İzometri.*

Teorem 16.3. *Lipschitz $\Rightarrow$ Üniform sürekli $\Rightarrow$ Sürekli.*

Kanıt İskeleti. $\delta = \varepsilon/K$ seçimi Lipschitz $\Rightarrow$ ünif. sürekli gösterir. $\square$

Teorem 16.4. *Kompakt metrik uzaydan metrik uzaya her sürekli fonksiyon üniform sürekli.*

16.3 Algoritmalar

16.3.1 classify_finite_metric_map — $O(|X|^2)$

Algorithm 21 ClassifyMap(X, d_1, Y, d_2, f)

```

1:  $K \leftarrow 0$ ; isometry  $\leftarrow$  True; similarity_ratio  $\leftarrow$  None
2: for each pair  $(x_1, x_2)$  with  $x_1 \neq x_2$  do
3:   ratio  $\leftarrow d_2(f(x_1), f(x_2))/d_1(x_1, x_2)$ 
4:    $K \leftarrow \max(K, \text{ratio})$ 
5:   if  $d_2(f(x_1), f(x_2)) \neq d_1(x_1, x_2)$  then
6:     isometry  $\leftarrow$  False
7:   end if
8:   if similarity_ratio is None then
9:     similarity_ratio  $\leftarrow$  ratio
10:  else if ratio  $\neq$  similarity_ratio then
11:    similarity_ratio  $\leftarrow$  None
12:  end if
13: end for
14: return MetricMapProfile(lipschitz_constant =  $K$ , isometry = isometry, ...)
```

Karmaşıklık: $O(|X|^2)$.

16.4 pytop API

```

1 from pytop.metric_spaces import FiniteMetricSpace
2 from pytop.metric_map_taxonomy import (
3     classify_finite_metric_map,
4     MetricMapProfile,
5 )
```

classify_finite_metric_map(fms1, fms2, f) $\rightarrow$ MetricMapProfile döner (Result değil).

MetricMapProfile alanları: isometry, similarity, similarity_ratio, lipschitz_constant, non_expansive, bijective, homeomorphism.

16.5 Örnekler

Örnek 5.1 — Özdeşlik: İzometri

```

1 f_id = {1: 1, 2: 2, 3: 3, 4: 4}
2 prof_id = classify_finite_metric_map(fms, fms, f_id)
3 print("isometry:", prof_id.isometry)
4 print("lipschitz_constant:", prof_id.lipschitz_constant)

```

```

isometry: True
lipschitz_constant: 1.0

```

Örnek 5.2 — $2\times$ Ölçekleme: Benzerlik

```

1 f_scale = {1: 2, 2: 4, 3: 6, 4: 8}
2 prof_scale = classify_finite_metric_map(fms_line1, fms_line2, f_scale
3 )
4 print("similarity:", prof_scale.similarity)
5 print("similarity_ratio:", prof_scale.similarity_ratio)
6 print("lipschitz_constant:", prof_scale.lipschitz_constant)

```

```

similarity: True
similarity_ratio: 2.0
lipschitz_constant: 2.0

```

Örnek 5.3 — Sabit Fonksiyon: $K = 0$

```

1 f_const = {1: 1, 2: 1, 3: 1, 4: 1}
2 prof_const = classify_finite_metric_map(fms, fms, f_const)
3 print("lipschitz_constant:", prof_const.lipschitz_constant)
4 print("bijective:", prof_const.bijective)

```

```

lipschitz_constant: 0.0
bijective: False

```

16.6 Alıştırmalar

Kodlama

- K1. $\{1, 2, 3\}$ üzerinde $f(1) = 2, f(2) = 3, f(3) = 1$ (döngü) fonksiyonunu ayrık metrikte `classify_finite_metric_map` ile sınıflandırın.
- K2. Öklid metriklili $\{0, 1, 2, 3, 4\}$ üzerinde $f(x) = x+1$ kaydırma fonksiyonunun Lipschitz sabitini hesaplayın.
- K3. Sabit fonksiyon $f(x) = 1$ için $K = 0$ olduğunu doğrulayın.

Teori

- T1. İzometri $\Rightarrow$ genişlemez $\Rightarrow$ Lipschitz ($K = 1$) zincirini ispatlayın.
- T2. Lipschitz $\Rightarrow$ üniform sürekli olduğunu $\delta = \varepsilon/K$ seçimi ile ispatlayın.

Ek A

Alıştırma Çözümleri

Bu ek, kılavuz alıştırmalarının tam çözümlerini içerir. Kodlama çözümlerindeki tüm çıktılar gerçek çalıştırmadan alınmıştır; teori çözümleri tam argüman verir. Pilot kapsamında Bölüm 4 ve Bölüm 6 çözümleri yer alır; diğer bölümler yaygınlaştırma aşamasında eklenecektir.

Dizin

Açık top Bkz. Bölüm 14.

Banach sabit-nokta teoremi Bkz. Bölüm 15.

Bağlantılı uzay Bkz. Bölüm 8.

Cauchy dizisi Bkz. Bölüm 15.

Heine-Borel teoremi Bkz. Bölümler 7, 15.

Homeomorfizma Bkz. Bölümler 4, 16.

İzometri Bkz. Bölüm 16.

Kompaktlık Bkz. Bölüm 7.

Lipschitz sabit Bkz. Bölüm 16.

Metrik aksiyomları Bkz. Bölüm 14.

Sayılabilirlik aksiyomları Bkz. Bölüm 9.

Tamlık Bkz. Bölüm 15.

Topoloji aksiyomları Bkz. Bölüm 4.

Urysohn fonksiyon teoremi Bkz. Bölüm 6.